

1.预处理&关键字 (22道)

1.1宏定义是在编译的哪个阶段被处理的?

答案：宏定义是在编译预处理阶段被处理的。

解读：编译预处理：头文件包含、宏替换、条件编译、去除注释、添加行号。

1.2写一个“标准”宏MIN，这个宏输入两个参数并返回较小的一个。

答案：

```
1 | #define MIN(A, B) ((A) <= (B)? (A) : (B))
```

解读：

(1) 注意这个题目要用三重条件操作符，在宏中要小心地把参数用括号括起来，并且整个宏也要用括号括起来，防止替换时出现错误。

(2) 注意若写“least = MIN(*p++, b);”这句代码会产生副作用，将*p++代入宏体，指针p会做两次自增操作。

1.3已知数组table，用宏求数组元素个数。

答案：

```
1 | #define COUNT(table) (sizeof(table) / sizeof(table[0]))
```

解读：sizeof(table)得到数组长度，sizeof(table[0])得到数组元素长度，两者相除即可得到数组元素个数。

1.4带参宏和函数的区别?

(1) 带参宏只是在编译预处理阶段进行简单的字符替换；而函数则是在运行时进行调用和返回。

(2) 宏替换不占运行时间，只占编译时间；而函数调用则占运行时间（分配单元、保留现场、值传递、返回）。

(3) 带参宏在处理时不分配内存；而函数调用会分配临时内存。

(4) 宏不存在类型问题，宏名无类型，它的参数也是无类型的；而函数中的实参和形参都要定义类型，二者的类型要求一致。

(5) 而使用宏定义次数多时，宏替换后源程序会变长；而函数调用不使源程序变长。

1.5内联函数的优缺点和适用场景是什么？

(1) 优点：内联函数与宏定义一样会在原地展开，省去了函数调用开销，同时又能做类型检查。

(2) 缺点：它会使程序的代码量增大，消耗更多内存空间。

(3) 适用场景：函数体内没有循环（执行时间短）且代码简短（占用内存空间小）。

1.6关键字volatile的作用是什么？给出三个不同的例子。

(1) 作用：告诉编译器不要去假设（优化）这个变量的值，因为这个变量可能会被意想不到地改变。精确地说就是，优化器在用到这个变量时必须每次都小心地重新读取这个变量的值，而不是使用保存在寄存器里的备份。

(2) 例子：

①并行设备的硬件寄存器（如：状态寄存器）。

②一个中断服务子程序中会访问到的非自动变量。

③多线程应用中被几个线程共享的变量（防止死锁）。

1.7如何用C语言实现读写寄存器变量？

答案：

```
1 | #define rBANKCON0 (*(volatile unsigned long *)0x48000004)
2 | rBankCON0 = 0x12;
```

解读：

(1) 由于是寄存器地址，所以需要先将其强制类型转换为 "volatile unsigned long *"。

3

(2) 由于后续需要对寄存器直接赋值，所以需要解引用。

1.8下面代码能不能编译通过？

```
1 | #define c 3  
2 | c++;
```

答案：不能。

解读：自增运算符++用于变量，3是常量。

1.9“在C语言中，凡是以#开头的都是预处理命令，同时预处理命令都是以#开头的”，这句话是正确的吗？

正确。

1.10预处理器标识#error的作用是什么？

答案：编译程序时，只要遇到 #error 就会跳出一个编译错误。

解读：当程序比较大时，往往有些宏定义是在外部指定的（如makefile），或是在系统头文件中指定的，当你不太确定当前是否定义了 XXX 时，可写如下预处理代码：

```
1 | #ifdef XXX  
2 | #error "XXX has been defined"  
3 | #else  
4 | ...  
5 | #endif
```

这样，如果编译时出现错误，输出了XXX has been defined，表明宏XXX已经被定义了。

1.11用预处理指令#define声明一个常数，用以表明1年中有多少秒（忽略闰年问题）。

答案:

```
1 | #define SECONDS_PER_YEAR (60 * 60 * 24 * 365)UL
```

解读:

3

- (1) 注意预处理器将为你计算常数表达式的值, 并且整个宏体要用括号括起来。
- (2) 注意这个表达式将使一个16位机的整型数溢出, 因此要用到无符号长整型符号UL, 告诉编译器这个常数是的无符号长整型数。

1.12关键字static的作用是什么?

- (1) static修饰局部变量时: ①改变了其存储位置, 存储在静态区; ②同时改变了其生命周期, 为整个源程序, 因此它只被初始化一次, 若没显式初始化则自动初始化为0。
- (2) static修饰全局变量时: 改变了其作用域, 只可以被文件内所用函数访问。
- (3) static修饰函数时: 改变了其作用域, 只可被这一文件内的其它函数调用。

1.13下面是关键字const的使用示例, 请说明它们的作用:

```
1 | (1) const int a;           // a是一个整形常量
2 |     int const a;          // a是一个整形常量
3 | (2) const int *a;         // a是一个指向整形常量的指针变量
4 |     int * const a;        // a是一个指向整形变量的指针常量
5 |     int const * const a = &b; // a是一个指向整形常量的指针常量
6 | (3) char *strcpy(char *strDest, const char *strSrc); // 参数在函数内部不会被修改
7 |     const int strcmp(char *source, char *dest);      // 函数的返回值不能被修改
```

答案: 以注释形式展示。

解读: const放在*前是修饰指向的对象, 放在*后则是修饰指针本身。

1.14一个参数既可以是const还可以是volatile吗? 一个指针可以是volatile吗? 下面的函数有什么问题?

```
1 | int square(volatile int *ptr)
2 | {
3 |     return *ptr * *ptr;
4 | }
```

- (1) 是的。一个例子是只读的状态寄存器, 它是volatile因为它可能被意想不到地改变, 它是const因为程序不应该试图去修改它。

(2) 是的。一个例子是当一个中服务子程序修改一个指向一个缓冲区的指针时。
 (3) 这个函数的目的是用来返指针*ptr指向值的平方，但是，由于*ptr指向一个volatile型参数，编译器将产生类似下面的代码：

```
1 | int square(volatile int *ptr)
2 | {
3 |     int a, b;
4 |     a = *ptr;
5 |     b = *ptr;
6 |     return a * b;
7 | }
```

由于*ptr的值可能被意想不到地该变，因此a和b可能是不同的。结果，这段代码可能返不是你所期望的平方值！正确的代码如下：

```
1 | long square(volatile int *ptr)
2 | {
3 |     int a;
4 |     a = *ptr;
5 |     return a * a;
6 | }
```

1.15关键字typedef 在C语言中频繁用以声明一个已经存在的数据类型的同义字。也可以用预处理器做类似的事。例如，思考一下下面的例子：

```
1 | #define dPS struct s *
2 | typedef struct s * tPS; //(顺序、分号、#号)
```

以上两种情况的意图都是要定义dPS 和 tPS 作为一个指向结构体s的指针。哪种方法更好呢？为什么？

(1) typedef更好。

(2) 举个例子：

```
1 | dPS p1, p2;
2 | tPS p3, p4;
```

第一行代码扩展为 struct s * p1, p2; 即定义p1为一个指向结构体的指针，p2为一个实际的结构体，这也许不是你想要的。第二行代码正确地定义了p3 和p4 两个指针。

1.16关键字sizeof的作用是什么？函数strlen()呢？

(1) sizeof关键字用来计算变量、数据类型所占内存的字节数。sizeof(数组名)得到数组所占字节数，sizeof(字符串指针名)得到指针所占字节数。

(2) 而strlen()函数则用来测试字符串所占字节数，不包括结束字符 '\0'。strlen(字符数组名)得到字符串所占字节数，strlen(字符串指针名)得到字符串所占字节数。

1.17 关键字extern的作用是什么？

答案：用于跨文件引用全局变量，即在本文件中引用一个已经在其他文件中定义的全局变量。

解读：

- (1) 注意引用时不能初始化，如extern var，而不能是extern var = 0。
- (2) 另外，函数默认是extern类型的，表明是整个程序（工程）可见的，加不加都一样。

1.18 extern"C"的作用？

答案：

- (1) 在C++代码中调用C函数，用法：extern "C">{C函数库头文件/函数声明}。
- (2) 在C代码中调用C++函数，用法：在C++的头文件中加extern"C">{头文件/函数声明}。

注意：extern"C"只能用于C++文件中。

1.19 关键字auto的作用是什么？

答案：用来定义自动局部变量，自动局部变量在进入声明该变量的语句块时被建立，退出语句块时被注销，仅在语句块内部使用。

解读：其实普通局部变量就是自动局部变量，只是省略了auto这一关键字。

1.20 关键字register的作用是什么？使用时需要注意什么？

- (1) 作用：编译器会将register修饰的变量尽可能地放在CPU的寄存器中，以加快其存取速度，一般用于频繁使用的变量。

(2) 注意：register变量可能不存放在内存中，所以不能用&来获取该变量的地址；只有局部变量和形参可以作为register变量；寄存器数量有限，不能定义过多register变量。

1.21 C语言编译过程中，关键字volatile和extern分别在哪个阶段起作用？

答案：volatile在编译阶段，extern在链接阶段。

解读：C语言编译过程分为预处理、编译、汇编、链接。

1.22 const与#define的异同？

(1) 异：const有数据类型，编译器可以做静态类型检查；而宏定义没有类型，可能会导致类型出错。

(2) 同：两者都可用来定义常数。

讨论

评论



牛客944597986号

1#

解答很详细!

发表于 2021-01-05 15:48:45

赞(1) 回复(0)



牛客685342718号

2#

老哥,定义秒数这个宏因为UL编译不过,我用c++要放到括号里面才可以.

发表于 2021-02-26 22:34:14

赞(0) 回复(0)



牛客176783230号

3#

解答详细!

发表于 昨天 02:47:50

赞(0) 回复(0)

2.数据类型 (22道)

2.1 用变量a给出下面的定义

- (1) 一个整型数： `int a`。
- (2) 一个指向整型数的指针（一重指针）： `int *a`。
- (3) 一个指向指针的的指针，它指向的指针是指向一个整型数的指针（二重指针）： `int **a`。
- (4) 一个有10个整型数的数组： `int a[10]`。
- (5) 一个有10个指针的数组，这10个指针是指向整型数的（指针数组）： `int *a[10]`。
- (6) 一个指向有10个整型数数组的指针（数组指针）： `int (*a)[10]`。
- (7) 一个指向函数的指针，该函数有一个整型参数并返回一个整型数（函数指针）：
`int (*a)(int)`。
- (8) 一个有10个指针的数组，这10个指针均指向函数，该函数有一个整型参数并返回一个整型数（函数指针数组）： `int (*a[10])(int)`。

2.2下面的代码输出是什么，为什么？

```

1  void foo(void)
2  {
3      unsigned int a = 6;
4      int b = -20;
5      (a + b > 6)? printf("> 6") : printf("<= 6");
6  }
```

答案：输出是“>6”。

解读：当运算表达式中存在有符号数和无符号数时，有符号数隐式转换成了无符号数（即底层的补码不变，但是此数从有符号数变成了无符号数）。注意，正数的补码为其本身，负数的补码为其反码+1。因此-20变成了一个非常大的正整数，所以该表达式计算出的结果“>6”。

2.3写出float x与“零值”比较的if语句。

答案：

```
1 | if(x > -0.000001 && x < 0.000001);
```

解读：因为计算机在处理浮点数的时候是有误差的，所以不能将浮点型变量用“==”或“!=”与数字比较，应该设法转化成“>”或“<”此类形式。

1

2.4下面代码有什么错误？

```
1 | #include<stdio.h>
2 | void main()
3 | {
4 |     char *s = "AAA";
5 |     s[0] = 'B';
6 |     printf("%s", s);
7 | }
```

(1) "AAA"是字符串常量，s是指针，指向这个字符串常量，所以声明s的时候就有问题，应该是const char* s="AAA"。

(2) 然后又因为是常量，所以对s[0]的赋值操作是不合法的。

2.5下面代码输出是什么？

```
1 | #include<stdio.h>
2 | void main()
3 | {
4 |     int *a = (int *)2;
5 |     printf("%d", a + 3);
6 | }
```

答案：输出是14。

解读：代码将数值2强制类型转换为int类型指针，int类型指针加3 相当于指向后面第三个int类型变量的首地址，一个int类型变量占4个字节，所以加3相当于指针往后移了12个字节，指向地址14处。

2.6下面代码运行后会是什么现象？

```
1 | #include<stdio.h>
2 | #define N 500
3 | void main()
4 | {
5 |     unsigned char count;
6 |     for(count = 0; count < N; count++)
7 |     {
8 |         printf("---%d---\n", count);
9 |     }
```

2/8

答案：进入不断打印count值的死循环。

解读：因为unsigned char 类型变量的最大值为255，所以count只能从0一直增加到255，然后又恢复为0，无法退出for循环。

1

2.7下面函数的返回值是？

```
1 | int foo(void)
2 | {
3 |     int i;
4 |     char c = 0x80;
5 |     i = c;
6 |     if(i > 0)
7 |         return 1;
8 |     return 2;
9 | }
```

答案：返回值为2。

解读：因为0x80 == 128，超出了char类型变量c的表示范围（-128~127），所以c == -128，进而i == -128，i < 0。

2.8结构体内存对齐原则？

答案：

(1) 第一个成员的首地址（地址偏移量）为0。

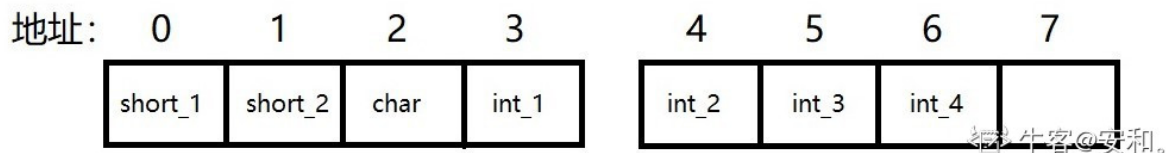
(2) 成员对齐：以4字节对齐为例，如果自身类型小于4字节，则该成员的首地址是自身类型大小的整数倍；如果自身类型大于等于4字节，则该成员的首地址是4的整数倍。若内嵌结构体，则内嵌结构体的首地址也要对齐，只不过自身类型大小用内嵌结构体的最大成员类型大小来表示。数组可以拆开看做n个数组元素，不用整体看作一个类型。

(3) 最后结构体总体补齐：以4字节对齐为例，如果结构体中最大成员类型小于4字节，则大小补齐为结构体中最大成员类型大小的整数倍；如果大于等于4字节，则大小补齐为4的整数倍。内嵌结构体也要补齐。

注意：32位编译器，一般默认对齐方式是4字节。

2.9结构体内存对齐的原因？

- (1) 平台原因（移植原因）：不是所有的硬件平台都能访问任意地址上的任意数据。
- (2) 性能原因：数据结构（尤其是栈）应该尽可能地在自然边界上对齐，因为访问未对齐的内存，处理器需要做两次内存访问，而访问对齐的内存仅需要一次。如下图所示，访问对齐的short变量只需要一次，而访问未对齐的int变量则需要访问两次。



2.10给定结构体，它在内存中占用多少字节（32位编译器）？

```
1 struct A
2 {
3     char t : 4;          // 4位
4     char k : 4;          // 4位
5     unsigned short i : 8; // 8位
6     unsigned long m;      // 4字节
7 };
```

根据结构体内存对齐原则，共占用8字节。

2.11在32位系统中，有如下结构体，那么sizeof(fun)的数值是？

```
1 #pragma pack(1)
2 struct fun
3 {
4     int i;    // 4字节
5     double d; // 8字节
6     char c;   // 1字节
7 };
```

答案：sizeof(fun)得到的结果是13。

解读：因为预处理语句"#prama pack(1)"将编译器的字节对齐数改为1了，根据结构体内存对齐原则，该结构体占用的字节数为13。

2.12数组首元素地址和数组地址的异同？

- (1) 异：数组首元素地址和数组地址是两个不同的概念。例如int a[10]，a的值是数组首元素地址，所以a+1就是第二个元素的地址，int类型占用4个字节，所以两者相差4。而&a是数组地址，所以&a+1就是向后移动(10*4)个单位，所以两者相差40。

(2) 同：数组首元素地址和数组地址的值是相等的。

2.13下面代码输出是什么？

1

```
1  #include<stdio.h>
2  void main()
3  {
4      int a[5] = {1, 2, 3, 4, 5};
5      int *ptr = (int *)(&a + 1);
6      printf("%d, %d", *(a + 1), *(ptr - 1));
7  }
```

答案：输出为2, 5。

解读：a是数组首元素地址，所以*(a + 1)就是第二个元素a[1]。&a是数组地址，所以&a + 1是整个数组结尾的下一个地址，*(ptr - 1)就是a[4]。

2.14判断下列表达式正确与否？

```
1  char str[2][3] = {"a", "b"};           // 正确，str是一个可存放两个字符串的字符串数组
2  char str[2][3] = {{1, 2}, {3, 4}, {5, 6}}; // 错误，行列不匹配
3  char str[] = {"a", "b"};               // 错误，字符数组不能存放字符串
4  char str[2] = {"a", "b"};              // 错误，字符数组不能存放字符串
```

答案：以注释形式展示。

注意：在C语言中字符用"括起来，而字符串用""括起来。

2.15查看下面代码，p[6]等于几？

```
1  int a[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 0};
2  int *p = &a[1];
```

答案：等于8。

解读：p是一个int类型指针，指向a[1]，p[6]表示p往后移了6个单位（每个单位4个字节）并解引用，因此p[6]等于8。

2.16下面代码的输出结果是什么？

```
1  #include<stdio.h>
2  void main()
3  {
```

```

4 |     char *str[] = {"ab", "cd", "ef", "gh", "ij", "kl"}; //指针数组
5 |     char *t;
6 |     t = (str + 4)[-1];
7 |     printf("%s", t);
8 | }

```

答案：输出"gh"。

1

解读：str表示数组首元素地址，str + 4表示数组第五个元素地址，(str + 4)[-1]表示在第五个元素地址的基础上往前移一个元素并解引用，因此输出是第四个元素。

2.17变长数组是什么？

在C99标准中，允许定义数组时[]中的值是整型变量或整型表达式，比如说下面的代码在支持C99标准的编译器中可编译通过：

```

1 | #include<stdio.h>
2 | void main()
3 | {
4 |     int n;
5 |     scanf("%d", &n);
6 |     int array[n];
7 | }

```

2.18 bool类型包含于哪个头文件？

答案：在C99标准中位于stdbool.h。

注意：C89标准中则不支持，需要自己定义，代码如下：

```

1 | #define TRUE 1
2 | #define FALSE 0
3 | typedef int bool;
4 | bool res = TRUE;

```

2.19结构体struct和联合体union的区别？

- (1) 两者最大的区别在于内存的使用。
- (2) 结构体各成员拥有自己的内存，各自使用且互不干涉，遵循内存对齐原则。
- (3) 联合体所有成员共用一块内存空间，并且同时只有一个成员可以得到这块内存的使用权。一个联合体变量的总长度应至少能容纳最大的成员变量，且需要进行内存补齐。

2.20 给了一个地址a，分别强转类型为：int变量、int指针、数组指针、指针数组、函数指针。

int变量	(int) a;
int指针	(int *)a;
数组指针	(int (*)[])a;
指针数组	(int *[])a;
函数指针	(int (*)(int))a;

2.21 执行完下面代码，c的值是多少？

```
1 unsigned int a = 1;
2 int b = 0;
3 int c = 0;
4 c = a + b > 0 ? 1 : 2;
```

答案：c的值是1。

解读：有符号数与无符号数一起运算时，有符号数转为无符号数。有符号数int b = 0的在内存中的补码是0，因此转为无符号数之后依然是0，a + b == 1 > 0，c == 1。

2.22 C语言中不同数据类型之间的赋值规则？

答案：

- (1) 整数与整数之间 (char, short, int, long) :
 - ①长度相等：内存中的数据不变，只是按不同的编码格式来解析。
 - ②长赋值给短：截取低位，然后按短整数的数据类型解析。
 - ③短赋值给长：如果都是无符号数，短整数高位补0；如果都是有符号数，短整数高位补符号数；如果一个有符号一个无符号，那么先将短整数进行位数扩展，过程中保持数

据不变，然后按照长整数的数据类型解析数据。

(2) 整数与浮点数之间

①浮点数转整数：截取整数部分。

②整数转浮点数：小数部分为0，整数部分与整数相等。

(3) float与double之间

①double转float会丢失精度。

②float转double不会丢失精度。

注意：整数在内存中都是以补码的形式存储的。

讨论

评论



意义非凡

1#

字符数组不能存放字符串吗，char str[]={“aaaaa”};这样没问题呀

发表于 2021-02-27 00:00:42

赞(0) 回复(2)

安和。 作者： 你是说2.4吗?字符数组可以放字符串，没问题，但题目2.4是字符串指针，也叫字符串常量，初始化之后就不能改变了

2021-02-27 08:09:06

赞(0) 回复(0)

队长_放下枪： 他说的是2.14吧，你的注释有问题

2021-02-28 12:10:13

赞(0) 回复(0)

请输入你的观点

回复

3.内存管理&编程题 (20道)

3.1由gcc编译的C语言程序占用的内存分为哪几个部分？

栈区(stack)	存放函数的参数、局部变量。
堆区(heap)	提供程序员动态申请的内存空间。
全局（静态） 区(static)	存放全局变量和静态变量，初始化不为0的全局变量和静态变量、const型常量在一块区域（.data段），未初始化的、初始化为0的全局变量和静态变量在相邻的另一块区域（.bss段）。
程序代码区	存放函数体的二进制代码和字符串常量。

3.2小端：一个数据的低位字节数据存储在低地址

大端：一个数据的高位字节数据存储在低地址

例如：int a=0x12345678; //a首地址为0x200，大端存储格式如下：

数据：	0x12	0x34	0x56	0x78
地址：	0x200	0x201	0x202	0x203

如何判读一个系统的大小端存储模式？

(1) 方法一：int *强制类型转换为char *，用“[]”解引用

```
void checkCpuMode(void)
{
    int c = 0x12345678;
    char *p = (char *)&c;
    if(p[0] == 0x12)
        printf("Big endian.\n");
    else if(p[0] == 0x78)
        printf("Little endian.\n");
    else
        printf("Uncertain.\n");
}
```

(2) 方法二：int *强制类型转换为char *，用“*”解引用

```

void checkCpuMode(void)
{
    int c = 0x12345678;
    char *p = (char *)&c;
    if(*p == 0x12)
        printf("Big endian.\n");
    else if(*p == 0x78)
        printf("Little endian.\n");
    else
        printf("Uncertain.\n");
}

```

(3) 方法三：包含short跟char的共用体

```

void checkCpuMode(void)
{
    union Data
    {
        short a;
        char b[sizeof(short)];
    }data;
    data.a = 0x1234;

    if(data.b[0] == 0x12)
        printf("Big endian.\n");
    else if(data.b[0] == 0x34)
        printf("Little endian.\n");
    else
        printf("uncertain.\n");
}

```

3.3全局变量和局部变量的区别？

(1) 全局变量储存在静态区，进入main函数之前就被创建，生命周期为整个源程序。

(2) 局部变量在栈中分配，在函数被调用时才被创建，在函数退出时销毁，生命周期为函数内。

3.4以下程序中，主函数能否成功申请到内存空间？

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
void getmemory(char *p)
{
    p = (char *)malloc(100);
    strcpy(p, "hello world");
}
int main()
{

```

```

    char *str = NULL;
    getmemory(str);
    printf("%s\n", str);
    free(str);
    return 0;
}

```

答案：不能。

解读：getmemory(str)没能改变str的值，因为传递给子函数的只是str的复制值NULL，main函数中的str一直都是 NULL。正确的getmemory()如下：

①传递的是二重指针，即str的指针

```

void getmemory(char **p)
{
    *p = (char *)malloc(100);
    strcpy(*p, "hello world");
}

```

②传递的是指针别名，即str的别名，C++中

```

void getmemory(char * &p)
{
    p = (char *)malloc(100);
    strcpy(p, "hello world");
}

```

3.5 请问运行下面的Test()函数会有什么样的后果？

```

void GetMemory(char **p, int num)
{
    *p = (char *)malloc(num);
}
void Test(void)
{
    char *str = NULL;
    GetMemory(&str, 100);
    strcpy(str, "hello");
    printf("%s\n", str);
}

```

答案：内存泄漏。

解读：调用malloc()申请内存空间，使用完毕之后没有调用free()释放内存空间并使指针指向NULL。

3.6 请问运行下面的Test()函数会有什么样的后果？

```

char *GetMemory(void)
{
    char p[] = "hello world";
    return p;
}
void Test(void)

```

```
{
    char *str = NULL;
    str = GetMemory();
    printf("%s\n", str);
}
```

答案：打印野指针内容，可能是乱码。

解读：GetMemory()返回的是指向栈内存的指针，但该栈内存已被释放，该指针的地址不是 NULL，成为野指针，新内容不可知。

3.7 请问运行下面的Test()函数会有什么样的后果？

```
void Test(void)
{
    char *str = (char *) malloc(100);
    strcpy(str, "hello");
    free(str);
    if(str != NULL)
    {
        strcpy(str, "world");
        printf("%s\n", str);
    }
}
```

答案：篡改堆区野指针指向的内容，后果难以预料，非常危险。

解读：

(1) free(str);之后，str成为野指针，没有置为NULL，if(str != NULL)语句不能阻止篡改操作。

(2) 野指针不是NULL指针，是指向被释放的或者访问受限的内存的指针。

(3) 造成野指针原因：①指针变量没有被初始化，任何刚创建的指针不会自动成为NULL；②指针被free或删除之后，没有置NULL；③指针操作超越了变量的作用范围，比如要返回指向栈内存的指针或引用，因为栈内存存在函数结束时会被释放。

3.8在C语言中memcpy和memmove是一样的吗？

答案：

(1) memcpy()与memmove()一样都是用来拷贝src所指向内存内容前n个字节到dest所指的地址上。

(2) 不同的是，当src和dest所指的内存区域重叠时，memcpy可能无法正确处理，而memmove()仍然可以正确处理，不过执行效率上略慢些。

解读：

(1) memcpy()无论什么情况下，都是从前往后拷贝内存。当源地址在前，目的地址在后，且两个区域有重叠时，会造成拷贝错误，达不到理想中的效果。

```
void *memcpy(void *dest, const void *src, size_t count)
{
    if(dest == NULL || src == NULL || count <= 0) return NULL;
    char *d = (char *)dest;
    char *s = (char *)src;
    while(count--)
    {
        *d++ = *s++;
    }
    return dest;
}
```

(2) memmove()则分两种情况：目的地址在前，源地址在后的情况下，从前往后拷贝内容。否则从后往前拷贝内容。无论什么情况都能达到理想中的效果。

```
void *memmove(void *dest, const void *src, size_t count)
{
    if(dest == NULL || src == NULL || count <= 0) return NULL;
    if(dest < src)
    {
        char *d = (char *)dest;
        char *s = (char *)src;
        while (count--)
        {
            *d++ = *s++;
        }
    }
    else
    {
        char *d = (char *)dest + count;
        char *s = (char *)src + count;
        while (count--)
        {
            *--d = *--s;
        }
    }
    return dest;
}
```

3.9 malloc的底层实现？

(1) malloc函数的底层实现是操作系统有一个由可用内存块连接成的空闲链表。调用malloc时，它将遍历该链表寻找足够大的内存空间，将该块一分为二（一块与用户

申请的大小相等，另一块为剩下来的碎片，会返回链表），调用free函数时，内存块重新连接回链表。

(2) 若内存块过于琐碎无法满足用户需求，则操作系统会合并相邻的内存块。

3.10在1G内存的计算机中能否malloc(1.2G)? 为什么?

(1) 有可能。

(2) 因为malloc函数是在程序的虚拟地址空间申请的内存，与物理内存没有直接的关系。虚拟地址与物理地址之间的映射是由操作系统完成的，操作系统可通过虚拟内存技术扩大内存。

3.11内存泄漏是什么?

(1) 内存泄漏 (Memory Leak) 是指程序中已动态分配的堆内存由于某种原因未释放或无法释放，造成系统内存的浪费，导致程序运行速度减慢甚至系统崩溃等严重后果。

(2) 分类：

①常发性内存泄漏：发生泄漏的代码会被多次执行到。

②偶发性内存泄漏：发生泄漏的代码在某些环境或操作下才会发生。

③一次性内存泄漏：只会被执行一次。

④隐式内存泄漏：程序在运行过程中不停地分配内存，直到结束时才释放。严格来讲这不算内存泄漏，但服务器运行时间很长，可能会耗尽所有内存。

3.12内存溢出是什么? 与内存泄漏有何关系?

(1) 内存溢出 (Out Of Memory) 是指应用系统中存在无法回收的内存或使用的内存过多，最终使得程序运行要用到的内存大于系统能提供的最大内存。此时程序无法运行，系统提示内存溢出。有时候会自动关闭软件。

(2) 造成内存溢出的原因：

①内存泄漏的堆积最终导致内存溢出。

②需要保存多个耗用内存过大的对象或加载单个超大的对象时，其大小超过了当前剩余的可用内存空间。

3.13堆栈溢出一般是由什么原因导致的？

- (1) 堆栈溢出一般包括堆内存溢出和栈内存溢出，两者都属于缓冲区溢出。
- (2) 堆内存溢出可能是堆的尺寸设置得过小/动态申请的内存没有释放。
- (3) 栈内存溢出可能是栈的尺寸设置得过小/递归层次太深/函数调用层次过深/分配了过大的局部变量。

3.14内存溢出和内存越界的区别？

- (1) 内存溢出：要求分配的内存超出了系统所能给予的，于是产生溢出。
- (2) 内存越界：向系统申请了一块内存，而在使用时超出了申请的范围，常见的是数组访问越界。

3.15位翻转

翻转前：

数：	v8	v7	v6	v5	v4	v3	v2	v1
位：	8	7	6	5	4	3	2	1

翻转后：

数：	v1	v2	v3	v4	v5	v6	v7	v8
位：	8	7	6	5	4	3	2	1

思路：目标数初始化为0，用&0x01的方式获得原始数的第1位，然后左移7位再与目标数按位或，接着原始数右移一位；再用&0x01的方式获得原始数的第2位，然后左移6位.....如此循环8次即可。最后返回目标数。

代码：

```
unsigned char bit_reverse(unsigned char input)
{
    unsigned char result = 0;
```

```

int bit = 8;
while(bit-->0)
{
    result |= ((input & 0x01) << bit);
    input >>= 1;
}
return result;
}

```

3.16字符串倒序：将一个字符串的字符顺序进行前后颠倒。

思路：双指针法，两个指针，一个指向字符串开头，另一个指向字符串结尾，相互交换指向的内容，接着头指针前进，尾指针后退，交换内容……直到两指针相遇。

代码：

```

#include<string.h>
void inverted_order(char *p)
{
    char *s1, *s2, tem;
    s1 = p;
    s2 = s1 + strlen(p) - 1;
    while(s1 < s2)
    {
        tem = *s1;
        *s1 = *s2;
        *s2 = tem;
        s1++;
        s2--;
    }
}

```

3.17找出一个字符串中一个最长的连续数字，并标注出位置和长度。

思路：指针法，指针从字符串开头开始寻找数字，若找到数字，则暂时记下位置，接着不断指向下一个字符，看连续的数字有多长，直到遇到非数字字符，然后比较长度是否比上一个连续数字长，若是则记录位置跟长度，接着寻找下一个数字，直到字符串结尾。最后返回最长的连续数字的位置和长度。

代码：

```

char *find(char *a, int *size)
{
    char *in = a, *temp, *pos;
    int count = 0, max = 0;
    while(*in != '\0')
    {
        if(*in >= '0' && *in <= '9') // 寻找数字
        {

```

```

        temp = in;
        while(*in >= '0' && *in <= '9') // 判断长度
        {
            count += 1;
            in++;
        }
        if(count > max) // 记录最长位置跟长度
        {
            pos = temp;
            max = count;
            count = 0;
        }
    }
    in++;
}
*size = max;
return pos;
}

```

3.18 写一个函数，判断输入参数是不是质数（素数）。

思路：

(1) 质数是指大于1的自然数中，除了1和它本身不再有其他因数的自然数。一个大于1的自然数不是质数就是合数，因此可以将问题转换为判断合数。

(2) 合数一定可以由两个自然数相乘得到，一个小于或等于它的平方根（大于1），另一个大于或等于它的平方根。因此可以判断“2 ~ 输入参数的平方根”中是否有能被输入参数整除的数，若有则该数是合数，若没有则该数是质数。

代码：

```

int IsPrime (unsigned int p)
{
    unsigned int i;
    if(p <= 1)
    {
        printf("请输入大于1的自然数。\\n");
        return -1;
    }
    for(i = 2; i <= sqrt(p); i++)
    {
        if(p % i == 0)
        {
            printf("该数不是质数。\\n"); // 是合数
            return 0;
        }
    }
    printf("该数是质数。\\n");
    return 0;
}

```

3.19大小端转化：对一个输入的整型数进行大小端存储模式转化

思路：大小端转化就是将一个整型数的低字节放到高字节，高字节放到低字节，跟前面的位翻转类似，只不过这里的单位是字节，因此需要将位翻转中的&0x01改为&0xFF，<< bit改为size * 8，>>= 1改为 >> 8。

代码：

```
int endian_convert(int input)
{
    int result = 0;
    int size = sizeof(input);
    while(size--)
    {
        result |= ((input & 0xFF) << (size * 8));
        input >>= 8;
    }
    return result;
}
```

3.20验证回文串：给定一个字符串，验证它是否是回文串，只考虑字符和数字字符，可以忽略字母的大小写。（回文串即左右对称的字符串，如"A man, a plan, a canal: Panama"）

思路：双指针法，一个指针指向字符串开头，另一个指向字符串结尾，两个指针都往中间移动并寻找字符和数字，并将字母统一转为小写，然后比较是否相同，若不相同则返回false，若相同则继续寻找.....如此循环，若直到两指针相遇都没返回false，则返回true。

代码：

```
bool isPalindrome(char * s)
{
    char *left = s, *right = s + strlen(s) - 1;

    if(strlen(s) == 0) return true;

    while(left < right)
    {
        while(!
        ((*left >= 'a' && *left <= 'z') || (*left >= 'A' && *left <= 'Z') || (*left >= '0' && *left <= '9')) && left < right) // 找到字母或数字
            left++;
        while(!
        ((*right >= 'a' && *right <= 'z') || (*right >= 'A' && *right <= 'Z') || (*right >= '0' && *right <= '9')) && right > left) // 找到字母或数字
            right--;

        if(left < right)
        {
            if((*left >= 'A' && *left <= 'Z')) // 若为大写，则转为小写
                *left = *left + 32;
            if((*right >= 'A' && *right <= 'Z')) // 若为大写，则转为小写
```

```
        *right = *right + 32;

    if(*left == *right) // 比较
    {
        left++;
        right--;
    }
    else
        return false;
}
else
    return true;
}

return true;
}
```

讨论

 评论

4.综合题 (18道)

4.1下面代码输出是几?

```
int main()
{
    int j = 2;
    int i = 1;
    if(i = 1) j = 3;
    if(i = 2) j = 5;
    printf("%d", j);
}
```

答案：输出为5。

解读：注意if的条件语句用的是赋值符“=”而不是等号“==”，因此条件一直为真。

4.2负数和正数的反码、补码分别是什么?

- (1) 负数的反码：对原码除符号位外的其余各位逐位取反就是反码。
- (2) 负数的补码：负数的补码就是对反码加1。
- (3) 正数的原码、反码、补码都一样。

4.3编译和链接有什么不同? (如对外部符号的处理)

- (1) 编译 (+汇编) 生成的是目标文件(*.o)。编译过程中对于外部符号 (如用extern跨文件引用的全局变量) 不做任何解释和处理，外部符号对应的就是“符号”。
- (2) 链接生成的是可执行程序。链接将会解释和处理外部符号，外部符号对应的是地址。

4.4函数参数的传递方式有几种?

- (1) 两种：值传递、指针传递。
- (2) 严格来看，只有一种传递，值传递，指针传递也是按值传递的，复制的是地址。

4.5局部变量能否和全局变量重名?

答案：能，局部会屏蔽全局。要用全局变量，需要使用“::”。

注意：对于有些编译器而言，在同一个函数内可以定义多个同名的局部变量，比如在两个循环体内都定义一个同名的局部变量，而那个局部变量的作用域就在那个循环体内。

4.6如何引用一个已经定义过的全局变量？

答案：可以用引用头文件的方式（不建议，可能会造成重复定义），也可以用extern关键字。

注意：

（1）如果用引用头文件方式来引用某个在头文件中声明的全局变理，假定你将那个变量写错了，那么在编译期间会报错。

（2）如果你用extern方式引用时，假定你犯了同样的错误，那么在编译期间不会报错，而在链接期间报错。

4.7全局变量可不可以定义在可被多个.C文件包含的头文件中？

答案：可以，但要将该全局变量定义为static全局变量，否则会重复定义，但此时没有达到一般意义上的全局变量的效果，而是静态全局变量。

注意：建议不要在头文件中定义变量，只做变量的声明。

4.8语句for(; 1 ;)有什么问题？它是什么意思？

在C语言中没问题，和while(1)相同，因为for循环中只要中间的表达式的值为TRUE即可。

4.9下面代码输出是几？

```
for(i = 0; i < 2, i < 3, i < 4; i++)  
    printf("%d \t", i);
```

答案：输出：0 1 2 3

解读：for中间的几条表达式中只要有一条为TRUE就可以了。

4.10 '0'的ASCII码为48，'A'的ASCII码为65，'a'的ASCII码为97。

4.11下面代码输出结果是什么？

```
void main()
{
    char *p1 = "name";
    char *p2;
    p2 = (char*)malloc(20);
    memset (p2, '0', 20);
    while(*p2++ = *p1++);
    printf("%s\n", p2);
}
```

答案：输出15个'0'。

解读：由于在语句“while(*p2++ = *p1++);”中，*p2每次接受*p1的赋值之后，都会进行一次自增操作，因此当p1指向的字符串共5个字符（包含'\0'）都赋值给*p2后，p2已经指向第六个字符'0'。

4.12指针只有在指向数组元素时才有意义吗？

不一定，比如说指向GPIO寄存器地址/字符串时。

4.13以下代码是否合乎语法？

```
int a[5];
*(2 + a) = 1;
a[2 - 1] = 1;
```

答案：是。

解读：

(1) 对指针解引用前，在其前面加上一个数字，并不会报错，只是解引用的地址会变。

(2) 访问数组时，其下标允许是常量表达式。

4.14一个32位的指针，如何按8字节的整数倍向下对齐，请写出代码。

答案：设该指针为p，则代码为：

```
p &= ~(8 - 1);
```

解读：

(1) 在存储的时候，为了提高效率，一般都会让地址偏移量落在2的m次方的位置上，而且经常有向上取整和向下取整两种需求。

(2) 向下取整：

```
#define PALIGN_DOWN(x, align) ((x) & ~((align) - 1))
```

(3) 向上取整：

```
#define PALIGN_UP(x, align) ((x) + ((align) - 1)) & ~((align) - 1)
```

4.15下面这段程序的运行结果？

```
int main()
{
    int a[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
    memcpy(a + 3, a, 5);
    for(int i = 0; i < 10; i++)
    {
        printf("%d ", a[i]);
    }
}
```

答案：输出为0 1 2 0 1 5 6 7 8 9

解读：因为memcpy的最后一个参数是需要拷贝的字节数目，而一个int类型的数据占4个字节，本题拷贝5个字节，实际上只能拷贝两个数字（从低位开始拷贝）。

4.16 gcc优化代码执行速度的编译选项是？

-O0/没有-O	不进行任何优化，用尽可能直接的方法来编译源代码，每行代码被直接转换为可执行文件中对应的指令。当调试一个程序时，这是最佳选项。
-O1/-O	使用能减少目标文件大小、执行时间，同时又不会使编译时间明显增加的代码，在编译大型程序的时候会显著增加编译时内存的使用。
-O2	使目标文件执行速度变快，但会增加编译时间。包含-O1的优化，同时执行了不会使目标文件变大、执行速度变慢的优化，不执行循环展开以及函数内联。
-O3	执行所有-O2优化选项，同时打开更深度的优化来提升执行文件的速度，比如函数内嵌，但也可能增加其大小。
-Os	专门优化目标文件的大小，执行所有不增加目标文件大小的-O2优化选项，并执行专门优化目标文件大小的选项。

4.17 new和malloc的区别？

- (1) new、delete、是C++中独有的操作符，而malloc、free是C/C++中的标准库函数。
- (2) C++允许重载new/delete操作符，而malloc/free是一个函数，不能重载。
- (3) new返回的是指定类型的指针，并且可以自动计算所申请内存的大小；而malloc返回的是void*类型，需要进行强制类型转换，并且需要指定要申请内存的大小。
- (4) new操作符从自由存储区上为对象动态分配内存空间，而malloc函数从堆上动态分配内存。
- (5) new内存分配失败时，会抛出bad_alloc异常；malloc内存分配失败时返回NULL。
- (6) 使用new创建对象在内存分配的时候会自动调用构造函数，同时也可以完成对对象的初始化，delete也能自动调用析构函数。

4.18指针与引用的相同和区别？

相同：都是地址的概念，指针是指内存的地址；引用是某块内存的别名，本质是指针常量，也是占内存的，有的编译器优化后就不占内存了。

不同：

- (1) 引用仅仅是一个别名，而指针是一个实体。
- (2) 引用只能在定义时被初始化一次，之后不可改变；非const指针可以改变。
- (3) 引用不能为空，指针可以为空。
- (4) 引用没有const，指针有const。
- (5) 引用使用时无需解引用，指针需要解引用。
- (6) 引用和指针自增（++）的意义不一样。
- (7) sizeof引用跟指针得到的大小不一样。



帖子还没人回复，快来抢沙发！

5.数据结构与算法 (17道)

5.1数组与链表的区别？

(1) 数组的元素个数在定义时就必须确定，且元素的类型必须一致；而链表的元素个数自由，且元素内可以有不同类型的数据。

(2) 数组的元素在内存中是按顺序存储的，而链表的元素是随机存储的。

(3) 要访问数组的元素可以按下标索引来访问，速度比较快；如果对它进行插入/删除操作的话，就得移动很多元素，所以对数组进行插入/删除操作效率很低。由于链表是随机存储的，如果要访问链表中的某个元素的话，那就得从链表的头逐个遍历，直到找到所需要的元素为止，所以链表的随机访问的效率就比数组要低；链表在插入/删除操作上有很高的效率（相对数组）。一句话总结就是：数组的访问效率高，而链表的插入/删除效率高。

5.2 $a = b * 2$; $a = b / 4$; $a = b \% 8$; $a = b / 8 * 8 + b \% 4$; $a = b * 15$;

效率最高的算法？

答案：

$a = b * 2$	$a = b << 1$;
$a = b / 4$	$a = b >> 2$;
$a = b \% 8$	$a = b \& 7$; // 7 = (0b111)
$a = b / 8 * 8 + b \% 4$	$a = ((b >> 3) << 3) + (b \& 3)$; // 3 = 0b11
$a = b * 15$	$a = (b << 4) - b$

解读：*、/、%分别可以用<<、>>、&来实现，效率更高。

5.3 C语言程序代码优化方法

(1) 选择合适的数据结构与算法；

(2) 使用尽量小的数据类型；

- (3) 使用自加、自减指令；
- (4) 用移位实现乘除法运算；
- (5) 求余运算用&（如a=a%8改为a=a&7）；
- (6) 平方运算用*（如a=pow(a,2.0)改为a=a*a）；
- (7) 延时函数的自加改为自减；
- (8) switch语句中根据发生频率来进行case排序；
- (9) 减少运算的强度。

5.4时间换空间、空间换时间的例子？

- (1) 时间换空间：冒泡排序。
- (2) 空间换时间：快速排序。

5.5什么是满二叉树、完全二叉树、平衡二叉树？

- (1) 当一个树每一层的结点个数都达到最大时，这个树是满二叉树。
- (2) 当一个树除了最后一层外其他每一层的结点数都达到最大，且最后一层的叶子结点都靠左排列时，这个树是完全二叉树。满二叉树是一种特殊的完全二叉树。
- (3) 当且仅当一个树两个子树的高度差不超过1时，这个树是平衡二叉树。

5.6堆和栈的区别？

数据结构的堆和栈	栈是一种先进后出的数据结构。堆是一种经过排序的树形数据结构（通常是二叉堆），每个结点都有一个值，根结点的值最小或最大，常用来实现优先队列，堆的存储是随意的。
C语言内存分配的堆和栈	栈是向下生长的，栈中分配函数参数和局部变量，其分配方式类似于数据结构中的栈。堆是向上生长的，堆中分配程序员申请的内存空间（一旦忘记释放会造成内存泄漏），其分配方式类似于数据结构中的链表。

5.7快慢指针有哪些应用？

判断链表是否有环	两个指针同时从链表的第一个节点出发，一个指针一次走一步，另一个指针一次走两步，如果走得快的指针追上走得慢的指针，则链表存在环；如果走得快的指针走到链表的末尾（NULL）都没有追上走得慢的指针，则链表不存在环。
找出链表的中间节点	两个指针同时从链表的第一个节点出发，一个指针一次走一步，另一个指针一次走两步，当走得快的指针走到最后一个节点时，走得慢的指针就刚好走

	到链表的中间节点。
删除链表倒数第n的节点	两个指针同时从链表的第一个节点出发，慢指针不动，快指针先走到第n个节点，然后两个指针开始一起走动，每次走一步，当快指针走到最后一个节点时，慢指针就处于链表的倒数第n个节点。
删除排序链表中的重复项	慢指针从第一个节点出发，快指针从第二个节点出发，两个指针一起走动，每次走一步，如果两个指针指向的节点数据相同，则释放快指针指向的节点，然后快指针指向下一个节点.....如此循环直到快指针指向末尾(NULL)。

5.8对线性表 (7,34,55,25,64,46,20,10) 进行散列存储时,若选用H (K) =K %9作为散列函数,则散列地址为1的元素有几个?

答案：4个，分别是：55，64，46，10。

解读：将线性表元素代入散列函数，即可得到散列地址。

5.9写一个程序, 要求功能：求出用1，2，5这三个数不同个数组合的和为100的组合个数。 如：100个1是一个组合，5个1加19个5是一个组合。

(1) 最容易想到的算法是暴力解法
思路： 设x是1的个数，y是2的个数，z是5的个数，number是组合数，注意到0 <= x <= 100，0 <= y <= 50，0 <= z <= 20。

代码：

```

void count(void)
{
    int x, y, z, number;
    number = 0;
    for (x = 0; x <= 100 / 1; x++)
    {
        for (y = 0; y <= 100 / 2; y++)
        {
            for (z = 0; z <= 100 / 5; z++)
            {
                if (x + 2 * y + 5 * z == 100)
                    number++;
            }
        }
    }
}

```

```
    printf("%d\t", number);  
}
```

(2) 上述暴力解法的时间复杂度为 $O(n^3)$ ，程序有些冗余，对其进行优化如下

思路：注意到上面代码的第三个for循环其实不是必要的，因为当1和2的数目确定了之后，加上一定数目的5能不能组成100也就确定了，没必要用for循环一个个去尝试，直接计算即可，优化后时间复杂度变为 $O(n^2)$ 。

代码：

```
void count(void)  
{  
    int x, y, z, number;  
    number = 0;  
    for (x = 0; x <= 100 / 1; x++)  
    {  
        for (y = 0; y <= 100 / 2; y++)  
        {  
            if (100 - x - y * 2 >= 0 && (100 - x - y * 2) % 5 == 0) // 判断能否5整除  
                number++;  
        }  
    }  
    printf("%d\t", number);  
}
```

5.10数组a[N]，存放了数字1 ~ N-1，其中某个数重复一次。写一个函数，找出被重复的数字，时间复杂度必须为 $O(N)$ 。

思路：数组有N个元素，刚好存放了数字1 ~ N - 1，那么是不是可以将数字与数组的下标对应起来存储，而重复的数字则刚好放在下标为0的位置。一开始假设a[0]是重复的数字，若不是则放到它该放的位置上，换另一个数字作为a[0]，若是则程序结束。如此循环，定能在N次循环内找到重复的数字。（哈希表思维，将元素的值与其存储位置关联起来）

代码：

```
int do_dup(int a[], int N)  
{  
    int temp;  
    // a[0]为监视哨  
    while (a[0] != a[a[0]]) // 若两者相等，则说明该数字是重复数字  
    {  
        temp = a[0]; // 若不相等，则将该数放到以该数为下标的位置上  
        a[0] = a[temp]; // 该位置原来的数则放到0为下标的位置上  
        a[temp] = temp;  
    }  
    return a[0]; // 返回重复数字  
}
```

5.11 写出快速排序的代码。

思路：数组一开始是乱序的，以第一个元素为基准，先将其保存；指针1从后往前寻找 $\leq$ 基准的元素，插在基准元素的位置，接着指针2从前往后寻找 $>$ 基准的元素，插在指针1的位置上；指针1继续往前寻找 $\leq$ 基准的元素，插在指针2的位置上，接着指针2继续往后寻找 $>$ 基准的元素，插在指针1的位置上；如此循环直到两指针相遇，将基准元素插在相遇的位置，至此， $\leq$ 基准的元素都在基准元素左边， $>$ 基准的元素都在基准元素右边，再递归地对左边的元素进行相同的操作，然后递归地对右边的元素进行相同的操作，即可完成排序，时间复杂度为 $O(n\log_2 n)$ 。（二分思维、递归思维）

代码：

```
void quick_sort(int *num, int start_num, int end_num)
{
    if(start_num < end_num)
    {
        int i = start_num;
        int j = end_num;
        int temp = num[start_num]; // 以第一个元素为基准
        while(i < j)
        {
            while(i < j && num[j] > temp) // 从后往前寻找比基准小的元素
                j--;
            if(i < j)
                num[i++] = num[j]; // 插在基准元素前面的空位上
            while(i < j && num[i] <= temp) // 从前往后找比基准大的元素
                i++;
            if(i < j)
                num[j--] = num[i]; // 插在基准元素后面的空位上
        }
        num[i] = temp; // 基准元素归位
        quick_sort(num, start_num, i - 1); // 递归地对基准元素左边的数据排序
        quick_sort(num, i + 1, end_num); // 递归地对基准元素右边的数据排序
    }
}
```

5.12 写出二分查找的代码。

思路：二分查找是对有序数组而言的，那么我们只需要拿中间的元素来跟目标数值比较，如果相等则查找完成；如果中间的元素小于目标数值，那么说明目标元素在右边；如果中间的元素大于目标数值，那么说明目标元素在左边。接着再拿右边或左边的中间元素来比较……如此循环直到找到目标元素/找不到退出循环。时间复杂度为 $O(n\log_2 n)$ （二分思维）

代码：

```
int binary_search(int array[], int value, int size)
{
    int low = 0;
    int high = size - 1;
    int mid;
    while(low <= high)
    {
        mid = (low + high) / 2;    // 二分
        if(array[mid] == value)    // 中间数据是目标数据
            return mid;
        else if(array[mid] < value) // 中间数据比目标数据小
            low = mid + 1;
        else    // 中间数据比目标数据大
            high = mid - 1;
    }
    return -1;
}
```

5.13单向链表逆序。

思路：尾插法，即先将第一个节点的next指针指向NULL，然后依次将第二个节点的next指针指向第一个节点，第三个节点的next指针指向第二个节点.....直至最后一个节点，并将指向最后一个节点的指针以头指针返回。

代码：

```
LNODE *link_reverse(LNODE *head)
{
    LNODE *pb, *pt;
    if(head == NULL)
        return head;
    pb = head -> next;
    head -> next = NULL;
    while(pb != NULL) // 尾插法
    {
        pt = pb -> next;
        pb -> next = head; // 插入新链表
        head = pb; // 新链表头指针
        pb = pt;
    }
    return head; // 返回头指针
}
```

5.14手写：字符串排序问题，时间复杂度O(n)，任意字符串的排序并统计重复的个数，例如输入：\$-#-#aeartvDEtGD%!%

输出：!1#1\$1%3-2D2E1G1a2e1r1t2v1

思路：字符的ASCII码总共只有128个（0~127），那么我们是不是可以建立一个长度为128的数组并初始化为0，然后遍历每个字符，并以字符的ASCII为下标将该元素+1。这样一来，只需要遍历一遍字符串，就可以将字符串排好序并统计好个数。接下来在遍历一遍这个数组，将对应的字符和个数打印出来即可。（哈希表思维，将元素的值与其存储位置关联起来）

代码：

```
#include <stdio.h>
#include <string.h>
void sort(char a[], int len)
{
    int i;
    char b[128] = {0}, key;
    for(i = 0; i < len; i++)
    {
        key = a[i]; // 将字符对应的ASCII码作为下标
        b[key]++; // 对应元素+1
    }
    for(i = 0; i < 128; i++)
    {
        if(b[i] != 0)
            printf("%c%d", i, b[i]); // 按顺序打印出字符跟个数
    }
    printf("\n");
}
```

5.15判断链表是否有环。

思路：双指针法，定义两个指针，同时从链表的头节点出发，一个指针一次走一步，另一个指针一次走两步。如果走得快的指针追上了走得慢的指针，那么链表就是环形链表；如果走得快的指针走到了链表的末尾（fast == NULL）都没有追上第一个指针，那么链表就不是环形链表。

代码：

```
bool IsLoop(NODE *head)
{
    if (head == NULL)
        return false;

    NODE *slow = head -> next; // 初始时，慢指针从第一个节点开始走1步
    if (slow == NULL)
        return false;

    NODE *fast = slow -> next; // 初始时，快指针从第一个节点开始走2步
    while (fast != NULL && slow != NULL) // 当单链表没有环时，循环到链表末尾结束
    {
        if (fast == slow) // 快指针追上慢指针
```

```

        return true;
    slow = slow -> next;    // 慢指针走一步
    fast = fast -> next;    // 快指针走两步
    if (fast != NULL)
        fast = fast -> next;
}
return false;
}

```

5.16已知两个已经从小到大排列的数组，将它们中所有的数组合成一个新的数组，要求新的数组也是按照从小到大排列的。

思路：原来的两个数组已经是顺序排列好的，那么为了减小时间复杂度，我们只需要新建一个数组，接着从后往前比较两个数组的元素的大小，并从后往前填充到新数组即可。时间复杂度为 $O(n)$ 。

代码：

```

int merge(int *array1, int len1, int *array2, int len2, int *array3)
{
    int len3 = len1 + len2;
    while(len1 > 0 && len2 > 0)
    {
        array3[len3-- - 1] = array1[len1 - 1] > array2[len2 - 1] ? array1[len1-- - 1] : array2[len2-- - 1]; // 比较数组元素大小并填充到新数组
    }
    while(len1 > 0) // 若数组1还有元素则依次填充
    {
        array3[len3-- - 1] = array1[len1-- - 1];
    }
    while(len2 > 0) // 若数组2还有元素则依次填充
    {
        array3[len3-- - 1] = array2[len2-- - 1];
    }
    return 0;
}

```

5.17写出二叉树的中序遍历代码。

思路：中序遍历，即先遍历左子树，然后访问根节点，最后遍历右子树，在遍历左子树跟右子树时，也是同样按照左、中、右的顺序。前序遍历和后续遍历则分别是中、左、右和左、右、中的顺序。由于每次都是同样的过程，因此可用递归实现。

代码：

```

void in_order_traversal(Node *node)
{
    if(NULL != node -> left)
        in_order_traversal(node -> left);
}

```

```
printf("%d\t", node -> data);  
if(NULL != node -> right)  
    in_order_traversal(node -> right);  
}
```

讨论

 评论



帖子还没人回复，快来抢沙发！

1.ARM处理器与中断 (15道)

1.1 CPU的内部结构?

CPU的内部结构大致可以分为:

- (1) 控制单元 (指令寄存器、指令译码器、操作控制器) 。
- (2) 运算单元 (算术逻辑单元) 。
- (3) 存储单元 (专用寄存器和通用寄存器)
- (4) 时钟。

1.2 CPU跟内存、虚拟内存、硬盘的关系?

- (1) CPU要调用的程序和数据来自硬盘, 但是CPU又不能直接读写硬盘上的系统、程序和数据, 所以必须先将硬盘的内容存储在内存中, 才能被CPU读写。
- (2) 因此内存是一个中转站, 对计算机的运行速度有较大影响。
- (3) 当系统需要的内存空间大于实际的物理内存空间时, 就需要用到虚拟内存了。虚拟内存可以将部分硬盘空间模拟成内存空间, 将暂时不运行的程序和不使用的数据存储在硬盘上, 需要时再将其存储到内存。

1.3 ARM结构处理器可分为哪几类?

嵌入式微处理器	32位以上的处理器, 具有较高的性能。
嵌入式微控制器	典型代表是单片机, 一般以某一种微处理器内核为核心, 芯片内部集成Flash、RAM、EEPROM、总线、总线逻辑、定时/计数器、串行接口等各种必要的功能模块。
嵌入式DSP	对系统结构和指令进行了特殊设计, 使其适合于执行DSP算法, 编译效率较高, 执行执行速度也较快。

1.4嵌入式微处理器和DSP有什么区别? 1/6

- (1) 嵌入式微处理器和DSP一个偏重控制、一个偏重运算。
- (2) 嵌入式微处理器外围接口丰富，标准化、通用性、功耗控制等做得很好，适用于消费电子、家用电器等控制领域。
- (3) DSP对系统结构和指令做了优化，能进行大量数据的快速计算，适用于音视频处理等领域。

1.5 ARM处理器有哪些工作状态？ARM指令和Thumb指令有什么区别？

答案：

- (1) ARM处理器共有ARM、Thumb/Thumb-2、调试三种状态。
- (2) ARM指令是32位的，较全面；Thumb指令是16位的，较精简。

解读：

ARM状态	工作于32位指令状态，所有指令均为32位。
Thumb状态	工作于16位指令状态，所有指令均为16位。
Thumb-2状态	ARM状态和Thumb状态是早期版本，近期推出的Thumb-2状态兼有16和32位指令，具有更高的性能、更低的功耗以及更少的内存占用。具有Thumb-2技术的ARM处理器无需在ARM和Thumb-2状态之间切换了。
调试状态	处理器停机调试。

1.6 RISC（精简指令集计算机）和CISC（复杂指令集计算机）的区别？

- (1) RISC控制器多采用硬件连线控制方式，以期更快的执行速度；而CISC控制器绝大多数采用微程序控制方式。
- (2) RISC只有加载和存储指令可以访问内存，数据处理指令只对寄存器的内容操作，为了加速程序的运算，RISC会设置多组寄存器，并指定特殊用途的寄存器，因此通用寄存器数量较CISC多。CISC架构允许数据处理指令对内存进行操作，因此需要的寄存器数量比较少。

(3) RISC大多指令在一个时钟周期内完成，且指令长度统一、数目少；而CISC的复杂指令通过CPU内的微码来完成，需要多个时钟周期，且指令长度不一、数目多。

(4) RISC在实现一个功能的时候，需要的指令数目多，编译器设计就更复杂；CISC的指令丰富，它的编译器可以少做很多事情。

(5) RISC较CISC更易实现指令流水线，因为其指令大多在一个时钟周期内完成。

1.7 ARM内部传输数据的总线有哪些？

ARM内部传输数据采用AMBA总线，共有四个版本：

- (1) AMBA1: ASB、APB;
- (2) AMBA2: AHB、APB;
- (3) AMBA3: AHB、AXI、ATB、APB;
- (4) AMBA4: AXI、ATB、ACE、APB。

1.8中断是嵌入式系统中重要的组成部分，这导致了很多编译开发商提供一种扩展：让标准C支持中断。具体表现是，产生了一个新的关键字__interrupt。下面的代码就使用了__interrupt关键字去定义了一个中断服务子程序(ISR)，这段代码有什么问题？

```
__interrupt double compute_area (double radius)
{
    double area = PI * radius * radius;
    printf(" Area = %f", area);
    return area;
}
```

答案：

- (1) 对于单片机来说，ISR不能传递参数。
- (2) 对于单片机来说，ISR不能有返回值。
- (3) 在许多的处理器/编译器中，浮点运算一般都是不可重入的。此外，ISR应该是短而有效率的，在ISR中做浮点运算是不明智的。
- (4) 与第三点一脉相承，printf()经常有重入和性能上的问题。

解读：重入一般可以理解为一个函数在同时多次调用，例如操作系统在进程调度过程中，或者单片机、处理器等的中断的时候会发生重入的现象。满足下面条件之一的多

数是不可重入函数：

- (1) 使用了静态数据结构。
- (2) 调用了malloc或free。
- (3) 调用了标准I/O函数，标准IO库很多实现都以不可重入的方式使用全局数据结构。
- (4) 进行了浮点运算。许多的处理器/编译器中，浮点一般都是不可重入的（浮点运算大多使用协处理器或者软件模拟来实现）。

1.9简述处理器中断处理的过程。

处理器在中断处理的过程中，一般分为以下几个步骤：中断请求 -> 中断响应 -> 保护现场 -> 中断服务 -> 恢复现场 -> 中断返回。

1.10保护现场、恢复现场的过程？

(1) 保护现场

- ①首先将R0~R7（快速中断）或R0~R12（其他中断） PUSH进堆栈中保存，并将堆栈指针保存在对应中断模式的R13（SP）中。
- ②把即将执行的下一条指令（PC-4）的地址保存到对应中断模式的R14（LR）中；把CPSR的值保存到对应中断模式的SPSR中，以实现对处理器当前状态、中断屏蔽及各标志位的保护。
- ③设置CPSR的相应位：设置CPSR[4:0]的5位以进入相应的工作模式；CPSR[5] = 0 切换到ARM状态；设置CPSR[7] = 1禁止IRQ中断；如果进入复位模式或FIQ模式，还要设置CPSR[6] = 1以禁止FIQ中断。
- ④给程序计数器PC强制赋值，转入中断向量地址，执行相应的中断服务程序。

(2) 恢复现场

- ①将原来保存在堆栈中的R0~R12或R0~R7 POP出栈，赋值给相应的寄存器。
- ②将LR的值赋值给PC，将SPSR的值赋值给CPSR中，恢复被中断的程序状态。

1.11复位中断与其他中断有什么不同？ 4/6

(1) 当中断产生后，复位中断立即中止当前指令的执行，其余情况都是当处理器完成当前指令后，再去响应中断。

(2) 如果是复位中断，系统自动从0x00000000开始重新执行程序，无需中断返回。

1.12什么是中断向量？什么是中断嵌套？

(1) 中断向量：中断服务子程序的入口地址。

(2) 中断嵌套：中断系统正在执行一个中断服务程序时，有另一个优先级更高的中断源提出请求，这时会暂停当前正在执行的级别较低的中断源的服务程序，处理级别更高的中断源。处理完毕后再返回到被中断了的中断服务程序。

1.13外部中断请求（IRQ）和快速中断请求（FIQ）的异常向量地址分别为？

0x00000018、0x0000001C。

1.14中断的优缺点？

(1) 优点：实现CPU和I/O设备的并行，提高CPU的利用率和系统的性能。

(2) 缺点：中断处理过程需要保护现场、恢复现场，整个过程需要一定的时间和空间开销。如果中断的频率太高，会降低系统的性能。

1.15中断服务程序能不能有参数和返回值？

(1) 在单片机裸机程序中，中断服务程序既不能有参数，也不能有返回值。

(2) 但是在带操作系统的嵌入式系统中，中断服务程序可以有参数，也可以有返回值。

讨论

评论



帖子还没人回复，快来抢沙发！

2.寄存器与存储器 (13道)

2.1 ARM的31个通用寄存器R0~R15中，程序计数器PC为R15、程序链接寄存器LR为R14、堆栈指针寄存器SP为R13。

2.2寄存器掉电会丢失数据吗？

寄存器是由触发器构成的，因此掉电会丢失数据。

2.3 NOR Flash与NAND Flash的区别？

	NOR Flash	NAND Flash
单位	字节	页（一般为512字节）
容量	一般为1~16MB	一般为8~128M
成本	较高	较低
性能	读速度比NAND Flash稍快	写入、擦除速度比NOR Flash快很多
接口	带有SRAM接口，有足够的地址引脚来寻址，可以很容易地存取其内部的每一个字节	使用复杂的I/O口来串行地存取数据，8个引脚用来传送控制、地址和数据信息
耐用性	最大擦写次数是十万次	最大擦写次数是一百万次
软件支持	写入和擦除都需要MTD（Memory Technology Devices，内存技术驱动程序），运行代码不需要任何软件支持	写入和擦除都需要MTD，运行代码也要需要MTD

2.4 SRAM、DRAM、SDRAM的区别？

- (1) SRAM：静态的随机存储器，加电情况下，不需要刷新，数据不会丢失，CPU的缓存就是SRAM。
- (2) DRAM：动态随机存储器，加电情况下，也需要不断刷新，才能保存数据，最为常见的系统内存。
- (3) SDRAM：同步动态随机存储器，即数据的读取需要时钟来同步，也可用作内存。

2.5磁盘跟硬盘的关系？

磁盘包括早期使用的软磁盘和现在经常使用的硬磁盘，硬磁盘简称硬盘。

2.6 RAM是什么？

- (1) RAM是随机存储器，速度很快，可随机读写，但断电则丢失数据，一般用作内存。
- (2) RAM的种类有很多，常见的有SRAM、DRAM、SDRAM。

2.7 ROM是什么？

- (1) ROM是只读存储器，速度较慢，不能直接与CPU进行交互，断电后数据不丢失，一般用来保存断电不丢失的程序。
- (2) 常见的ROM有PROM、EPROM、EEPROM。

2.8 SRAM跟DRAM的区别？

	SRAM	DRAM
存储原理	触发器	电容
是否刷新	不需要	需要
运行速度	快	慢

存储成本	高	低
发热量	大	小
送行列地址	同时送	分两次送
破坏性读出	否	是
集成度	低	高

2.9 ARM在不同工作模式时使用的寄存器有所不同，但共同点是？

- (1) R0~R7为公用的通用寄存器。
- (2) CPSR为公用的当前程序状态寄存器。
- (3) R15为公用的程序计数器PC。

2.10 CPU要先对cache做什么，才能读取DMA数据？

答案：CPU要先对cache做一个invalidate（作废）操作，再从内存中读取数据到缓存，保证缓存和内存中数据的一致性，才能读取DMA数据。

解读：

- (1) 为了正确进行DMA传输，必须进行必要的缓存操作，缓存操作主要分为invalidate（作废）和writeback（写回）。
- (2) DMA从外设读取数据供CPU使用时，可先进行invalidate（作废）操作。这样一来，处理器在读取缓存中的数据前，会先从内存中读取数据到缓存，保证缓存和内存中数据的一致性。
- (3) DMA初始设置由处理器提供数据时，可先进行writeback（写回）操作。这样一来，可以在DMA传输之前将缓存的数据写回到内存中，保证内存和缓存中数据的一致性。

2.11大小端各自的优点？用在何处？

答案：

- (1) 大端优点：符号位在低地址的第一个字节中，便于快速判断数据的正负和大小。
- (2) 小端优点：低地址放低字节，所以在强制类型转换的时候不需要调整字节的内容。而且CPU做数值运算时从内存中依次从低到高取数值直到最后刷新最高位的符号位，这样的运算方式更高效。
- (3) x86系列芯片采用小端模式，ARM芯片默认为小端模式，但可以切到大端。

注意：大小端是由CPU架构决定的，不是软件决定的。

2.12缓冲技术的作用是？

- (1) 改善CPU与I/O设备间速度不匹配的矛盾。
- (2) 提高CPU和I/O设备之间的并行性，提高系统的吞吐量和设备的利用率。
- (3) 减少对CPU中断的频率，放宽对中断响应时间的限制。

2.13缓冲技术的种类？

单缓冲、双缓冲、多缓冲、缓冲池。

讨论

 评论



帖子还没人回复，快来抢沙发！

3.I/O接口 (17道)

3.1 GPIO口一般有哪三个寄存器?

以S3C2410为例, 有控制寄存器、数据寄存器、上拉寄存器。

3.2 GPIO的输入输出模式有哪些?

答案:

- (1) 输入模式: 浮空输入、带上拉输入、带下拉输入、模拟输入。
- (2) 输出模式: 开漏输出、推挽输出、开漏复用输出、推挽复用输出。

解读:

浮空输入_IN_FLOATING	浮空输入, 可以做KEY识别。
带上拉输入_IPU	IO内部上拉电阻输入。
带下拉输入_IPD	IO内部下拉电阻输入。
模拟输入_AIN	应用ADC模拟输入, 或者低功耗下省电。
开漏输出_OUT_OD	IO输出0接GND, IO输出1则悬空, 需要外接上拉电阻才能输出高电平。当输出为1时, IO口的状态由上拉电阻拉高, 但同时IO口也可由外部电路拉低, 可以通过输入数据寄存器读取电平变化, 实现C51的IO双向功能。
推挽输出_OUT_PP	IO输出0接GND, IO输出1接VCC, 读输入值是未知的。
复用功能的推挽输出_AF_PP	片内外设功能 (I2C的SCL,SDA) 。
复用功能的开漏输出_AF_OD	片内外设功能 (TX1,MOSI,MISO.SCK.SS)

3.3请说明总线接口UART、USB、SPI、IIC的异同点。

总线接口	串/并	同步/异步	工作方式	速率	线路	距离	大小端
UART	串行	异步	全双工	慢，最快只有1.5Mbps	3线：RX、TX、GND	远，RS-485可达1200m	小端模式
USB	串行	同步	半双工	快，USB3.0可达5Gbps	4线：Vbus、GND、D+、D-	近，不超过5m	小端模式
SPI	串行	同步	全双工	快，可达50Mbps	3线或4线：SCLK、SIMO、SOMI、SS	远，可达10m	大端模式
IIC	串行	同步	半双工	慢，最快只有3.4Mbps	2线：SCL、SDA	近，不超过30cm	大端模式

3.4什么是异步串行？什么是同步串行？

(1) 异步串行方式的特点：

- ①以字符为单位传送信息，相邻两个字符间的间隔是任意长；即字符内部各位同步，字符间异步；
- ②因为一个字符中的比特位长度有限，所以接收时钟和发送时钟只要相近就可以了。

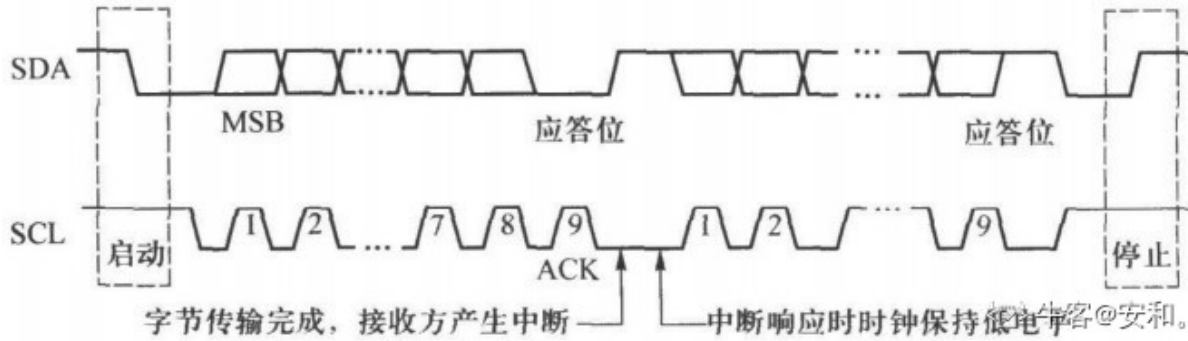
(2) 同步串行方式的特点：

- ①以数据块为单位传送信息，在一个数据块内，字符与字符之间无间隔；即字符内部各位同步，字符间也同步；

②因为一次传输的数据块中包含的数据较多，所以接收时钟与发送时钟需要严格同步。

3.5 IIC总线时序图？

(1) 时序总结：



总线空闲状态	SCL和SDA均为高电平，接上拉电阻。
启动信号（START）	在SCL保持高电平期间，SDA由高电平被拉低。由主控制器发出。
数据位传送（DATA）	在SCL保持高电平期间，SDA上的电平保持稳定，低电平为数据0、高电平为数据1。用法：主控制器和被控器都可发出。
应答信号（ACK）	在SCL保持高电平期间，SDA保持低电平。IIC总线上所有数据都是以8位字节传送的，发送器每发送一个字节，就在第9个时钟脉冲期间释放SDA（高电平），由接收器反馈一个ACK。
非应答信号（NACK）	在SCL保持高电平期间，SDA保持高电平。如果接收器是主控制器，则它在收到最后一个字节后，发送一个NACK，通知被控器结束数据发送，并释放SDA（高电平），以便主控制器发送一个STOP。
停止信号（STOP）	在SCL保持高电平时间，SDA由低电平被释放（拉高）。由主控制器发出。

(2) 写数据步骤：

- ①主机发起一个启动信号（START）。
- ②主机发送7bit从机地址+1bit读写选择位，1表示读、0表示写。
- ③从机产生应答信号（ACK）。
- ④主机发送8bit从机寄存器地址。
- ⑤从机产生应答信号（ACK）。
- ⑥主机发送一个字节数据。

- ⑦从机产生应答信号 (ACK) 。
- ⑧主机发送一个停止信号 (STOP) 。

(3) 读数据步骤

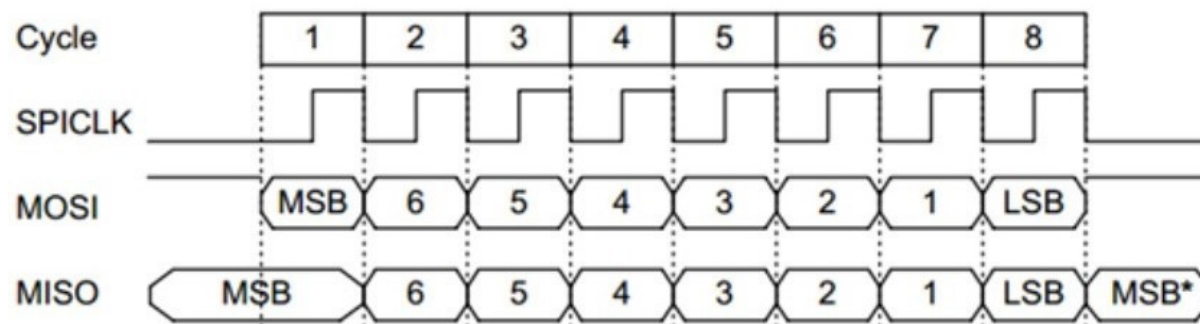
- ①主机发送一个启动信号 (START) 。
- ②主机发送7bit从机地址+1bit读写选择位, 1表示读、0表示写。
- ③从机产生一个应答信号 (ACK) 。
- ④主机发送8bit从机寄存器地址。
- ⑤从机产生一个应答信号。
- ⑥主机再次发送一个启动信号 (START) 。
- ⑦主机再次发送7bit从机地址+1bit读写选择位, 1表示读、0表示写。
- ⑧从机产生一个应答信号 (START) 。
- ⑨主机读取一个字节数据。
- ⑩主机产生一个非应答信号 (NACK) 。之后产生一个停止信号 (STOP) 。

3.6 SPI的四种操作时序分别是什么？

SPI的时钟极性CPOL和时钟相位CPHA可以分别为0或1，由此构成了四种组合：

- ①CPOL = 0, CPHA = 0: 空闲时SCLK为低电平，在第一个边沿开始采样。

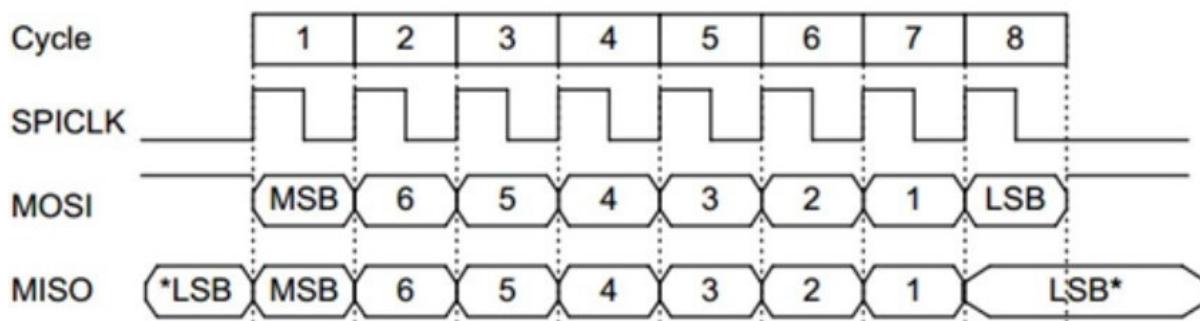
CPOL = 0, CPHA = 0 (Format A)



* MSB of character just received https://blog.csdn.net/qq_41410961/article/details/106471177 牛客@安和。

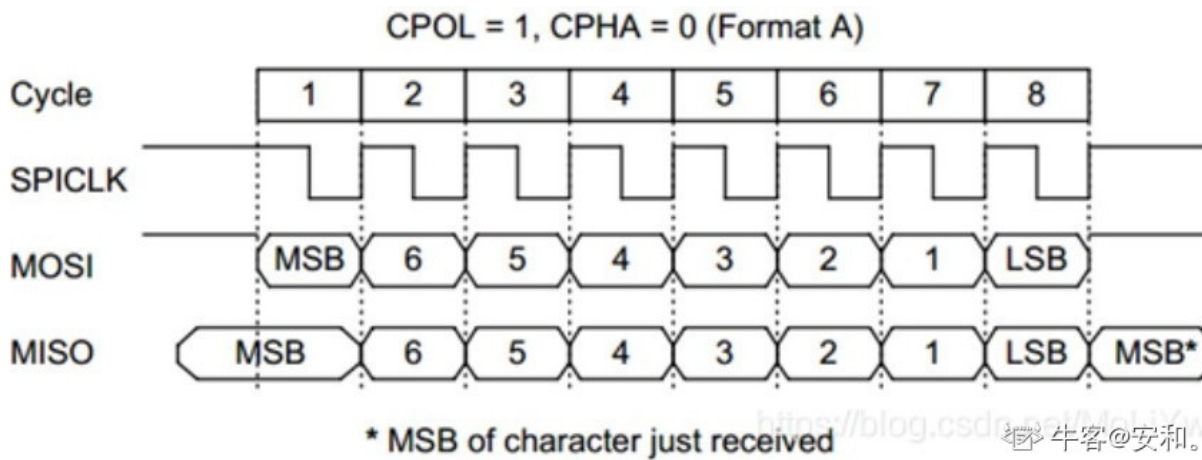
- ②CPOL = 0, CPHA = 1: 空闲时SCLK为低电平，在第二个边沿开始采样。

CPOL = 0, CPHA = 1 (Format B)

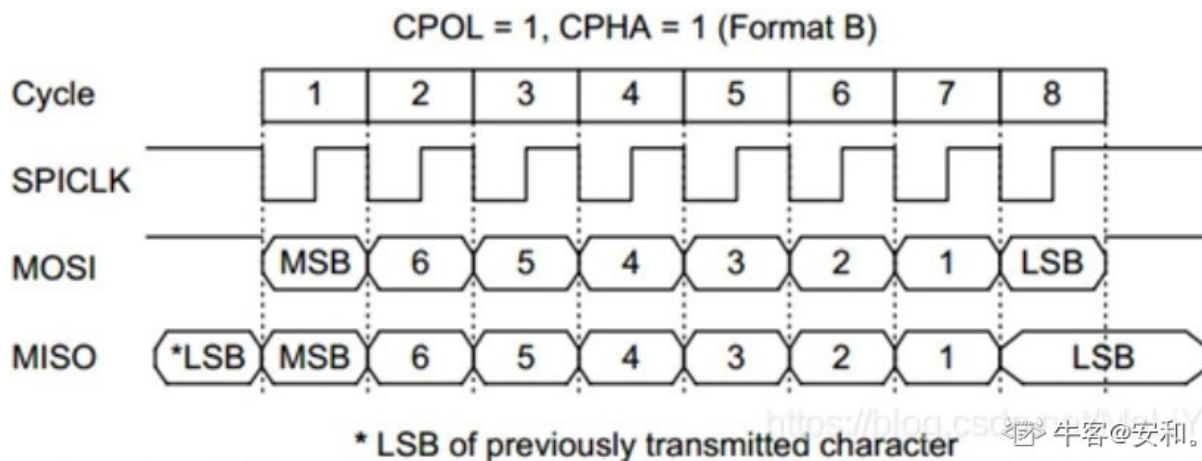


* LSB of previously transmitted character https://blog.csdn.net/qq_41410961/article/details/106471177 牛客@安和。

- ③CPOL = 1, CPHA = 0: 空闲时SCLK为高电平，在第一个边沿开始采样。



④CPOL = 1, CPHA = 1: 空闲时SCLK为高电平，在第二个边沿开始采样。



3.7哪种总线方式是全双工类型？哪种总线方式的传输距离较短？

- (1) UART、SPI是全双工类型。
- (2) IIC、USB的传输距离较短。

3.8编写一个通用型串口接收程序，如何实现判断未知设备的端口号和波特率？

- (1) 判断端口号：首先搜索计算机可用端口，然后逐个打开，分别定时1.5秒，有数据进来则认为是目标端口。
- (2) 波特率判断：将本机波特率分别设置为2400、4800、9600、19200、38600、57600、115200，每个波特率定时1.5秒，对收到的ASCII码进行分析，将小于等于127的认为是正常的字符，将大于127的认为是乱码，当正常字符数目大于等于10倍乱码数目则认为当前波特率为正确波特率，并终止往后搜索。另外，也可以通过示波器判断。

3.9串口如何发送浮点型数据？

可将多字节的浮点型数据分解成单字节逐个发送，接收后组合还原。

(1) 使用共用体来分解

```
union
{
    float f;
    unsigned long l;
}data_TX;
data_TX.f = 123.456;
```

(2) 发送时发送4字节（小端模式，低位先发）

```
TX = (unsigned char)data_TX.l;          // 低8位
TX = (unsigned char)(data_TX.l >> 8);
TX = (unsigned char)(data_TX.l >> 16);
TX = (unsigned char)(data_TX.l >> 24); // 高8位
```

(3) 接收端也定义同样的结构体，并接收4字节数据

```
union
{
    float f;
    unsigned long l;
}data_RX;
data_RX.l = RX;          // 低8位
data_RX.l |= RX << 8;
data_RX.l |= RX << 16;
data_RX.l |= RX << 24; // 高8位
```

(4) 最后data_RX.f == data_TX.f。

3.10 UART和TTL、RS-232、RS-485的关系？

答案：

- (1) UART是一种具有协议特征的收发器/接口/总线，也就是说它是一个按照特定协议来收发数据的硬件，它规定了数据按照什么格式和时序来传输。
- (2) 而TTL、RS-232、RS-485是三种不同的电气协议，是对电气特性的规定，作用于数据传输通路，但它并不包含对数据的处理方式。UART可以使用TTL电平，也可以使用RS-232、RS-485电平。

解读：三种电气协议的区别：

TTL（晶体管-晶体管逻辑电平）	规定+5V（或≥2.4V）等于逻辑“1”，0V（或≤0.4V）等于逻辑“0”，噪声容限为0.4V。
	6／9

RS-232	采用负逻辑传输，规定-5V ~ -15V等于逻辑“1”，+5V ~ +15V为逻辑“0”，噪声容限为2V。
RS-485	采用差分传输，规定A线电平比B线电平高200mV以上时为逻辑“1”，A线电平比B线电平低200mV以上时为逻辑“0”。

3.11 RS-232与RS-485的区别和联系？

(1) 区别：

- ①抗干扰性：RS-485接口的抗干扰性比RS-232接口强，因为RS-485采用差分传输。
- ②传输距离：RS-485接口（1200m）的传输距离比RS-232接口（50m）远。
- ③通信能力：RS485接口在总线上允许连接多达128个收发器，而RS-232接口只允许一对一通信。
- ④传输速率：RS-485接口的最高传输速率为10Mbps，而RS-232接口为20Kbps。
- ⑤信号线：RS-485接口组成半双工网络需要两根信号线，组成全双工网络需要四根信号线；RS-232接口一般使用RXD、TXD、GND三根线组成全双工网络。
- ⑥电气电平值：RS-485接口规定A线电平比B线电平高200mV以上时为逻辑“1”，A线电平比B线电平低200mV以上时为逻辑“0”。RS-232接口规定-5V ~ -15V等于逻辑“1”，+5V ~ +15V为逻辑“0”，噪声容限为2V。

(2) 联系：

- ①都可通过DB-9连接器连接。
- ②均可采用屏蔽双绞线传输。
- ③都是串行通信。
- ④通信方式都是全双工（一般RS-485是半双工）。

3.12 uart如何保证数据传输的正确性？

- (1) 在数据位的两端添加了起始位、奇偶校验位、停止位等用于数据的同步和纠错。

(2) 在接收端用16倍波特率对数据进行采样，取中间的采样值，很大程度上确保了采样数据的正确性。

3.13 保证数据无串扰，参数设置正确，但是串口A传输float类型给串口B，总是出现错误，请问可能是什么原因？

可能是发送端和接收端的大小端不一致。

3.14 什么是MSB，什么是LSB？

(1) MSB(Most Significant Bit): 最高有效位，是指二进制中最高值的比特。

(2) LSB(Least Significant Bit): 最低有效位，是指二进制中最低值的比特。

3.15 UART一帧可以传5/6/7/8位，IIC必须是8位，SPI可以8/16位。

3.16 请列举常用的串行通信方式（两种以上），并简述串行通信和并行通信不同之处、优缺点。

(1) 异步通信（UART）和同步通信（SPI、IIC、USB等）。

(2) 串行速度慢但占用资源少、线间干扰小；并行速度快但占用资源多、线间干扰相对大。

3.17 CAN总线接口相对于RS-232接口、RS-485接口的优点是什么？

(1) CAN总线接口相对于RS-232接口的优点是抗干扰能力强、传输距离远。它采用差分传输，内置CRC校验，传输可靠性强。

(2) CAN总线接口相对于RS-485接口的优点是能构成多主系统，同一时刻可以有两个或两个以上的设备处于发送状态，适用于实时性要求高的工控领域。

讨论

 评论



帖子还没人回复，快来抢沙发！

1.进程&线程（16道）

1.1异步IO和同步IO区别？

答案：如果是同步IO，当一个IO操作执行时，应用程序必须等待，直到此IO执行完。相反，异步IO操作在后台运行，IO操作和应用程序可以同时运行，提高系统性能，提高IO流量。

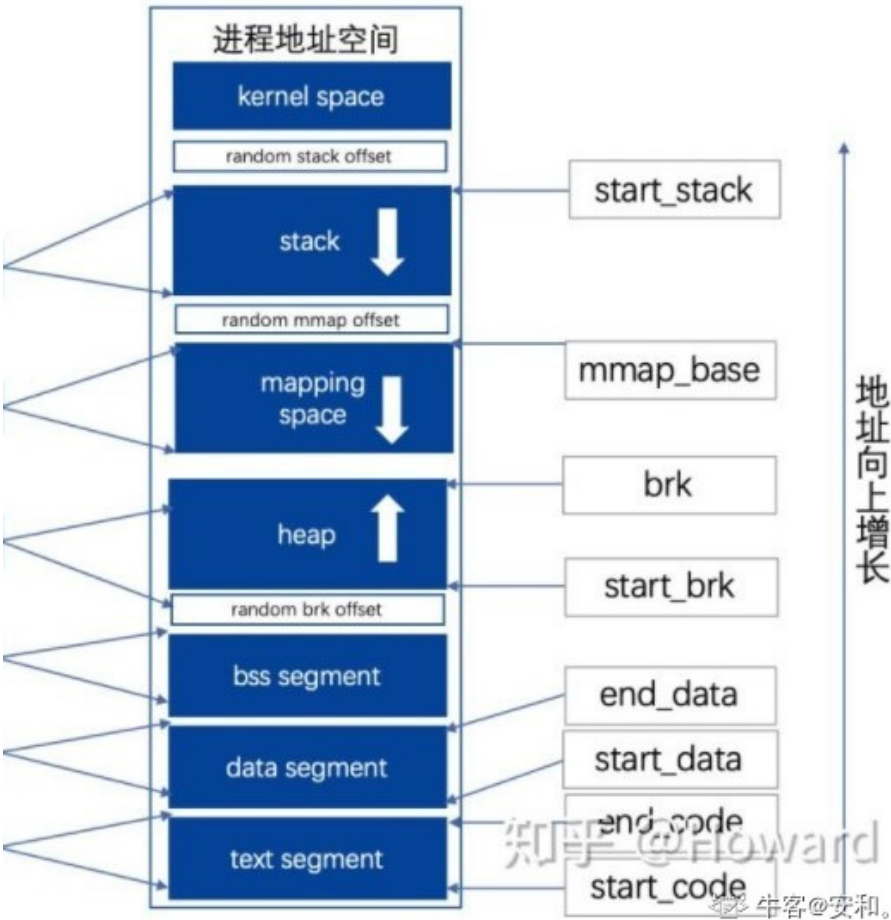
解读：在同步文件IO中，线程启动一个IO操作然后就立即进入等待状态，直到IO操作完成后才醒来继续执行。而异步文件IO中，线程发送一个IO请求到内核，然后继续处理其他事情，内核完成IO请求后，将会通知线程IO操作完成了。

1.2进程间通信方式？

管道 (pipe)	是一种半双工的通信方式，数据只能单向流动，而且只能在具有亲缘关系的进程之间使用，通常是父子进程。
有名管道 (named pipe)	也是半双工的通信方式，但是它允许用于无亲缘关系的进程之间的通信。
信号量 (semaphore)	是一个计数器，通常作为一种同步机制，用于进程和线程间的同步。
消息队列 (message queue)	是一个消息链表，存放在内核中并且由消息队列标识符标识。消息队列克服了信号传递信息少、管道只能承载无格式字节流以及缓冲区大小受限的缺点。
共享内存 (shared memory)	一段能够被多个进程共同访问的内存，由一个进程创建。共享内存是最快的IPC方式，它是针对其他进程间通信方式运行效率低而设计的，往往与其他通信方式如信号量配合使用，来实现进程间同步与通信。
套接字(socket)	可用于不同主机间的进程通信。
	1 / 7

信号(signal)	用于通知接收进程某个事件已经发生，是一种比较复杂的通信方式。
------------	--------------------------------

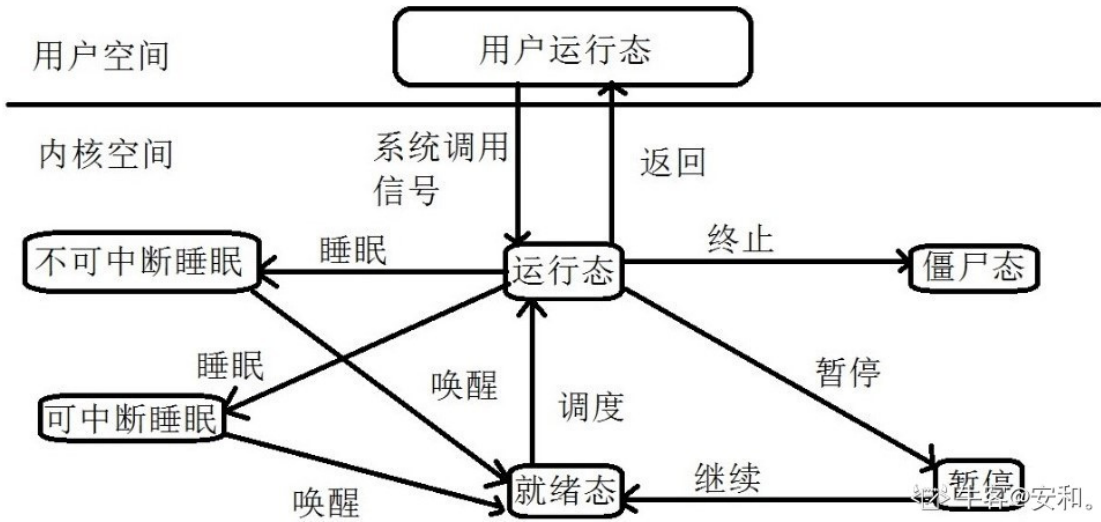
1.3进程的地址空间模型？



text segment	存储代码的区域。
data segment	存储初始化不为0的全局变量和静态变量、const型常量。
bss segment	存储未初始化的、初始化为0的全局变量和静态变量。
heap（堆）	用于动态开辟内存空间。
memory mapping space (内存映射区)	mmap系统调用使用的空间，通常用于文件映射到内存或匿名映射（开辟大块空间），当malloc大于128k时（此处依赖于glibc的配置），也使用该区域。在进程创建时，会将程序用到的平台、动态链接库加载到该区域。
stack（栈）	存储函数参数、局部变量。

kernel space	存储内核代码。
--------------	---------

1.4进程的五种状态分别是？



- (1) 就绪态：所有运行条件已就绪，只要得到了CPU时间就可运行。
- (2) 运行态：得到CPU时间正在运行。
- (3) 僵尸态：进程已经结束了但父进程还没来得及回收。
- (4) 等待态：包括浅度睡眠跟深度睡眠。进程在等待某种条件，条件成熟后即进入就绪态。浅度睡眠时进程可以被信号唤醒，但深度睡眠时必须等到条件成熟后才能结束睡眠状态。
- (5) 暂停态：暂时停止参与CPU调度（即使条件成熟），可以恢复。

1.5子进程从父进程继承的资源有哪些？

答案：用户号和用户组号、目录（当前目录、根目录）、打开的文件描述符、堆栈、共享内存。

注意：父进程和子进程拥有独立的地址空间和PID参数、自己的文件描述符、不同的父进程号。

1.6什么是进程上下文、中断上下文？

- (1) 进程上文：是指进程由用户态切换到内核态时需要保存用户态时CPU寄存器中的值，进程状态以及堆栈上的内容。即保存当前进程的状态，以便再次执行该进程

时，能够恢复切换时的状态，继续执行。

(2) 进程下文：是指切换到内核态后执行的程序，即进程运行在内核空间的部分。

(3) 中断上文：硬件通过中断触发信号，导致内核调用中断处理程序，进入内核空间。这个过程中，硬件的一些变量和参数也要传递给内核，内核通过这些参数进行中断处理。中断上文可以看作硬件传递过来的这些参数和内核需要保存的一些其他环境（主要是当前被中断的进程环境）。

(4) 中断下文：执行在内核空间的中断服务程序。

1.7如何防止僵尸进程过多导致系统空间爆满？

答案：每当子进程退出，父进程都会收到SIGCHLD信号，故可在父进程中设置SIGCHLD信号的捕获函数，在捕获函数中回收子进程。

```
void* handler(int sig)
{
    int status;
    if(waitpid(-1, &status, WNOHANG) >= 0)
    {
        printf("child is die\n");
    }
}
int main()
{
    signal(SIGCHLD, handler);
    int pid = fork();
    if(pid > 0) //父进程循环等待
    {
        while(1)
        {
            sleep(2);
        }
    }
    else if(0 == pid)
    {
        //子进程说自己die后就结束生命周期，之后父进程就收到SIGCHLD
        //信号调用handler函数接收结束子进程，打印child is die。
        printf("i am child, i die\n");
        exit(0);
    }
}
```

解读：

(1) 僵尸进程是当子进程比父进程先结束，而父进程又没有回收子进程，释放子进程占用的资源，此时子进程将成为一个僵尸进程。

(2) 可用top命令查看僵尸进程的数量 (zombie) , 也可用ps -aux | grep Z (僵尸进程的状态显示为Z) 来查看僵尸进程的PID等信息。

1.8 /proc目录下，以数字命名的目录表示什么？

以数字命名的目录表示当前一个运行的进程，目录名即为进程PID，其内的目录和文件给出了一些关于该进程的信息。

1.9进程和线程有什么区别？

- (1) 进程是系统中程序执行和资源分配的基本单位，线程是CPU调度的基本单位。
- (2) 一个进程个拥有多个线程，线程可以访问其所属进程地址空间和系统资源（数据段、已经打开的文件、I/O设备等），同时也拥有自己的堆栈。
- (3) 同一进程中的多个线程可以共享同一地址空间，因此它们之间的通信实现也比较简单，而且切换开销小、创建和消亡的开销也小。而进程间的通信则比较麻烦，而且进程切换开销、进程创建和消亡的开销也比较大。

1.10什么时候用多进程？什么时候用多线程？

- (1) 当程序的安全性、稳定性要求较高时用多进程。
- (2) 需要频繁通信/切换程序/创建跟销毁程序时用多线程。

1.11线程可以独立运行吗？一个线程崩溃会导致整个进程崩溃吗？

答案：线程不能独立运行，但一个线程崩溃不一定导致整个进程崩溃。

解读：

- (1) 线程属于进程，线程的运行需要依赖进程的地址空间和系统资源。
- (2) 线程崩溃的本质就是内存出错，若出错的内存没有被其他线程访问，则不会导致其他线程出错，也就不会导致进程崩溃。

1.12写出下列线程、互斥锁、信号量相关代码。

定义一个线程ID变量	pthread_t tid;
创建线程	pthread_create(&tid, NULL, pthread_func, NULL);
等待子线程结束，并回收资源	pthread_join(tid, NULL);
	5 / 7

与当前进程分离	<code>pthread_detach(tid);</code>
退出调用线程	<code>pthread_exit(NULL);</code>
取消线程	<code>pthread_cancel(tid);</code>
创建互斥锁	<code>pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;</code>
初始化一个互斥锁	<code>pthread_mutex_init(&mutex, NULL);</code>
对互斥锁上锁	<code>pthread_mutex_lock(&mutex);</code>
对互斥锁解锁	<code>pthread_mutex_unlock(&mutex);</code>
定义一个信号量	<code>sem_t sem;</code>
创建信号量并初始化它的值	<code>sem_init(&sem, 0, 1);</code>
信号量的值减1	<code>sem_wait(&sem);</code>
信号量的值加1	<code>sem_post(&sem);</code>

1.13线程间通信和同步方式有哪些？

信号、信号量、互斥锁、条件变量、自旋锁、读写锁。

1.14如何防止同时产生大量的线程？

- (1) 方法是使用线程池。
- (2) 线程池可以提高调度效率和限制资源使用，线程池中的线程达到最大数时，其他线程就会排队等候。

1.15互斥锁与信号量的区别？

- (1) 信号量用于线程同步，互斥锁用于线程互斥。
- (2) 信号量可以为非负整数，可以实现多个同类资源的多线程同步；互斥锁只能为0/1，只能用于一个资源的互斥访问。
- (3) 信号量可以由一个线程释放，另一个线程得到；互斥锁的加锁和解锁必须由同一线程分别对应使用，且多个线程使用多个互斥锁必须注意统一顺序，否则可能造成死锁。

1.16一个进程可以创建多少个线程？和什么有关？

- (1) 一个进程创建线程的个数由虚拟内存的大小和分配给线程的调用栈大小决定。一个线程的栈大小可通过- (2) 假设一个线程需要占用10M内存，那么在3G的地址空间上大概可以创建300个线程。

讨论

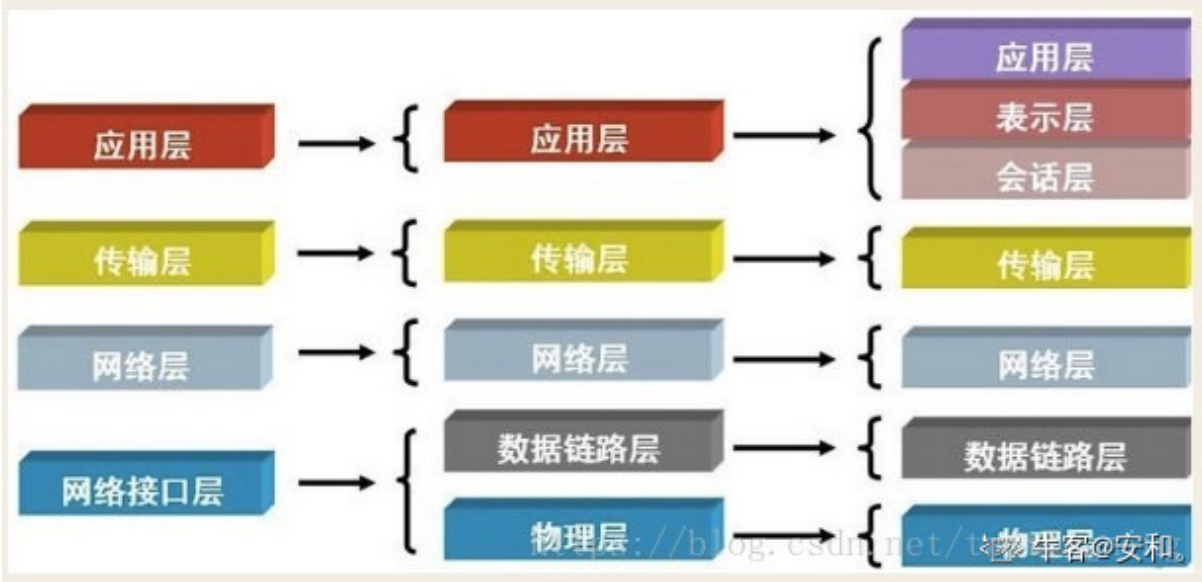
 评论



帖子还没人回复，快来抢沙发！

2.网络编程 (29道)

2.1列举一下OSI协议的各种分层。说说你最熟悉的一层协议的功能。



- (1) 七层划分为：应用层、表示层、会话层、传输层、网络层、数据链路层、物理层。
- (2) 五层划分为：应用层、传输层、网络层、数据链路层、物理层。
- (3) 四层划分为：应用层、传输层、网络层、网络接口层。（TCP/IP协议对应模型）
- (4) 各层功能：

应用层	在实现多个应用进程相互通信的同时，完成一系列业务处理所需的服务，比如电子邮件、文件传输、远程登录等。
传输层	为通信双方的主机提供端到端的服务，有两个不同的传输协议TCP和UDP，TCP提供可靠交付，而UDP并不能保证可靠交付。
网络层	处理分组在网络中的活动，例如分组的选路。
网络接口层	处理与电缆（或其他任何传输媒介）的物理接口细节。

2.2 TCP/IP协议包括？

应用层	Telnet（远程登录服务）、FTP（文件传输，使用TCP）、SMTP（建立于1
-----	--

	FTP上的邮件服务)、DNS (域名与IP地址相互转换) 等
传输层	UDP (无连接、不可靠)、TCP (面向连接、可靠传输)
网络层	IP (为主机提供一种无连接、不可靠、尽力而为的数据服务)、ICMP (主机与路由器之间传递控制信息)、IGMP (主机与路由器之间进行组播成员信息交互)
网络接口层	ARP (IP 地址-> MAC地址)、RARP (MAC地址 -> IP地址) 等

2.3 TCP通信建立和释放的过程？端口的作用？

- (1) 连接是三次握手，释放是四次挥手。
- (2) 端口是一个软件结构，被客户进程或服务进程用来发送和接收信息。一个端口对应一个16比特的数。服务进程通常使用一个固定的端口。

2.4 IP地址转换成物理地址的协议？反之？

答案：

- (1) 将IP地址转换成物理地址的协议是ARP（地址解析协议）。
- (2) 反之则是RARP（反地址解析协议）。

解读：

- (1) ARP协议工作流程：
 - ①首先，每台主机都会在自己的ARP缓冲区(ARP Cache)中建立一个ARP列表，以表示IP地址和MAC地址的对应关系。
 - ②当源主机需要将一个数据包要发送到目的主机时，会首先检查自己ARP列表中是否存在该IP地址对应的MAC地址，如果有，就直接将数据包发送到这个MAC地址；如果没有，就向本地网段发起一个ARP请求的广播包，查询此目的主机对应的MAC地址。此ARP请求数据包里包括源主机的IP地址、硬件地址、以及目的主机的IP地址。
 - ③网络中所有的主机收到这个ARP请求后，会检查数据包中的目的IP是否和自己的IP地址一致。如果不相同就忽略此数据包；如果相同，该主机首先将发送端的MAC地址

和IP地址添加到自己的ARP列表中，如果ARP表中已经存在该IP的信息，则将其覆盖，然后给源主机发送一个ARP响应数据包，告诉对方自己是它需要查找的MAC地址；

④源主机收到这个ARP响应数据包后，将得到的目的主机的IP地址和MAC地址添加到自己的ARP列表中，并利用此信息开始数据的传输。如果源主机一直没有收到ARP响应数据包，表示ARP查询失败。

(2) RARP协议工作流程：

①网络上的每台设备都会有一个独一无二的硬件地址，通常是由设备厂商分配的MAC地址。PC1从网卡上读取MAC地址，然后在网络上发送一个RARP请求的广播数据包，请求RARP服务器回复该PC的IP地址。

②RARP服务器收到了RARP请求数据包，为其分配IP地址，并将RARP回应发送给PC1。

③PC1收到RARP回应后，就使用得到的IP地址进行通讯。

2.5 IP地址的编码分为哪两部分？

IP地址由两部分组成，网络号和主机号。不过是要和“子网掩码”按位与上之后才能区分哪些是网络位哪些是主机位。

2.6应用程序ping发出的是什么报文？

答案：应用程序ping发出的是ICMP请求报文。

解读：ping的原理是利用网络上机器IP地址的唯一性，给目标IP地址发送一个数据包，通过对方回复的数据包来确定两台网络机器是否连接相通，时延是多少。

2.7 socket编程的流程？

(1) 服务器端流程：

函数	作用
socket()	创建套接字

bind()	绑定本地IP地址和端口号
listen()	设置监听队列长度
accept()	等待连接
read()	接收信息
close()	关闭套接字

(2) 客户端流程：

函数	作用
socket()	创建套接字
connect()	发送连接请求
write()	发送信息
close()	关闭套接字

2.8 epoll是什么？

(1) epoll是Linux网络编程中用于处理大批量文件描述符的机制，是对select/poll的改进。

(2) select监听的fd是有上限的，32位处理器一般为1024；且select/poll每次调用会遍历所有fd，时间复杂度为 $O(n)$ ，效率太低。而epoll监听的fd数量没有限制，且能在 $O(1)$ 的时间复杂度内完成操作。

(3) epoll相关的系统调用有：epoll_creat、epoll_ctl、epoll_wait/epoll_pwait（可屏蔽特定信号），分别用来创建一个epoll文件描述符、添加/删除/修改需要侦听的文件

描述符及事件、接收被侦听描述符的IO事件。epoll文件描述符用完之后直接close关闭即可。

2.9 TCP、UDP的区别？

- (1) TCP是面向连接的，UDP是面向无连接的。
- (2) TCP是面向字节流的，UDP是基于数据报的。
- (3) TCP提供可靠服务（正确性、顺序性），UDP提供不可靠服务。
- (4) TCP程序结构复杂，占用资源多；UDP程序结构简单，占用资源少。
- (5) TCP有拥塞控制；UDP没有拥塞控制，因此网络出现拥塞不会使源主机的发送速率降低，适合于实时应用，如IP电话、实时视频会议。
- (6) TCP只支持一对一；UDP支持一对一、一对多、多对一、多对多。

2.10 TCP、UDP的优缺点？

(1) TCP

优点：稳定可靠。在传输数据之前，会有三次握手来建立连接；在数据传输时，有校验和、确认、重传、序列号、窗口、拥塞控制机制；在数据传输结束后，还会有四次挥手来断开连接，节约系统资源。

缺点：效率低、占用系统资源多、易受攻击。建立连接和确认、重传、窗口、拥塞控制机制都会消耗大量时间；每台设备维护连接都会占用CPU、内存等资源；三次握手机制、确认机制导致TCP易受利用和攻击。

(2) UDP

优点：效率高、占用系统资源较少、比TCP稍安全。没有各种复杂机制，传输速度快，占用系统资源较少，被利用的漏洞也少一些。

缺点：不稳定、不可靠。没有各种复杂机制，网络质量不好时很容易丢包。

2.11 TCP、UDP使用场景？

(1) TCP：对网络质量有要求时，比如HTTPS、HTTP、FTP等文件传输协议，POP、SMTP等邮件传输协议。

(2) UDP：对网络质量要求不高时，要求网络通讯速度要快的场景，比如在线视频、网络语音电话、广播通信。

2.12 TCP为什么是可靠连接？

因为TCP传输的数据满足四大条件：不出错、不丢失、不重复、不乱序，而且拥有窗口机制、拥塞控制机制来提高传输效率。

2.13 TCP如何保证可靠传输？

(1) 校验和：发送数据报的二进制相加然后取反，检测数据在传输过程中的变化，有差错则丢弃。

(2) 确认应答：接收方收到正确的报文就会确认。

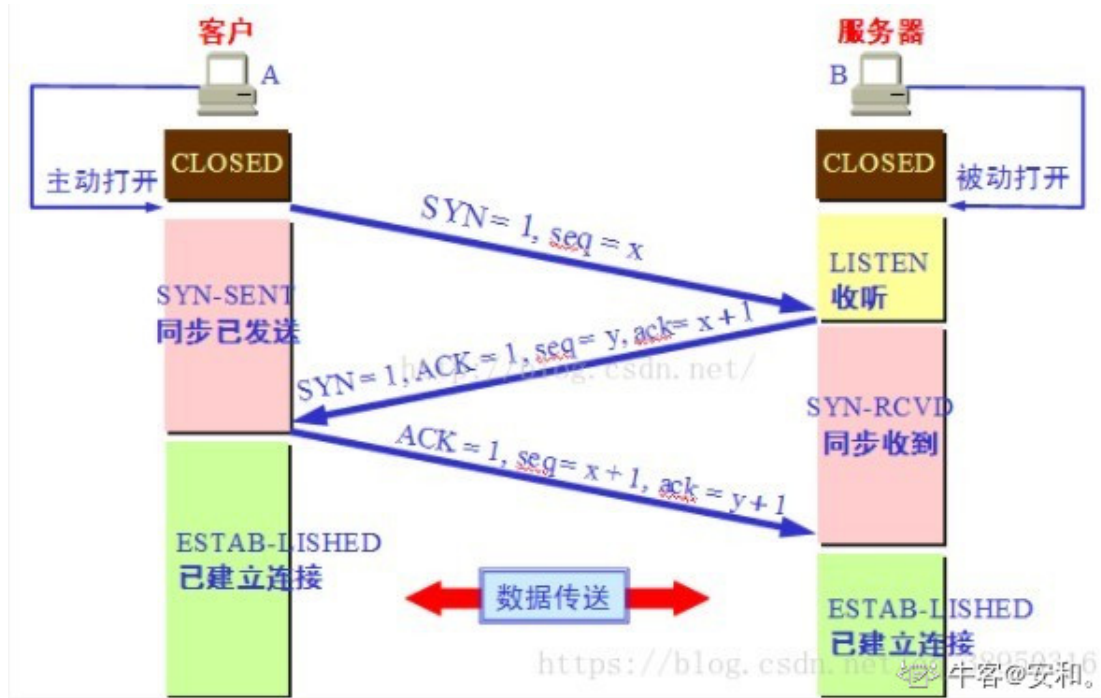
(3) 超时重传：发送方等待一定时间后没有收到确认报文则重传。

(4) 序列号：发送方对每一个数据包编号，接收方对数据包排序，保证不乱序、不重复。

(4) 窗口机制（流量控制）：双方会协调发送的数据包大小，保证接收方能及时接收。

(5) 拥塞控制机制：如果网络拥塞，发送方会降低发送速率，降低整个网络的拥塞程度。

2.14简述三次握手的过程。



(1) 最初两端的TCP进程都处于CLOSED状态，A主动打开链接而B被动打开链接。B的TCP服务器先创建传输控制块TCB，然后服务器进程就处于LISTEN状态，等待客户端的连接请求。若有则作出响应。

(2) 第一次握手：客户端创建传输控制块，然后向服务器发出连接请求报文（将标志位SYN置1，随机产生一个序列号 $seq=x$ ），接着进入SYN-SENT状态。

(3) 第二次握手：服务器收到请求报文后由 $SYN=1$ 得到客户端请求建立连接，回复一个确认报文（将标志位SYN和ACK都置1， $ack=x+1$ ，随机产生一个序列号 $seq=y$ ），接着进入SYN-RCVD状态。此时操作系统为该TCP连接分配TCP缓存和变量。

(4) 第三次握手：客户端收到确认报文后，检查 ack 是否为 $x+1$ ，ACK是否为1，是则发送确认报文（将标志位ACK置1， $ack=y+1$ ，序列号 $seq=x+1$ ），此时操作系统为该TCP连接分配TCP缓存和变量。服务器收到确认报文并检查无误后则连接建立成功，两者都进入ESTABLISHED状态，完成三次握手。

2.15为什么需要三次握手，第三次握手去掉行不行？

- (1) 不行。
- (2) 首先，第三次握手是为了防止已经失效的连接请求报文突然传输到了服务器，从而建立错误的连接，浪费资源。
- (3) 其次，第三次握手还能防止发生死锁，因为若为两次握手且服务器发出第二次握手而客户端没有收到，服务器开始传输数据报后客户端便不会理会，导致服务器以

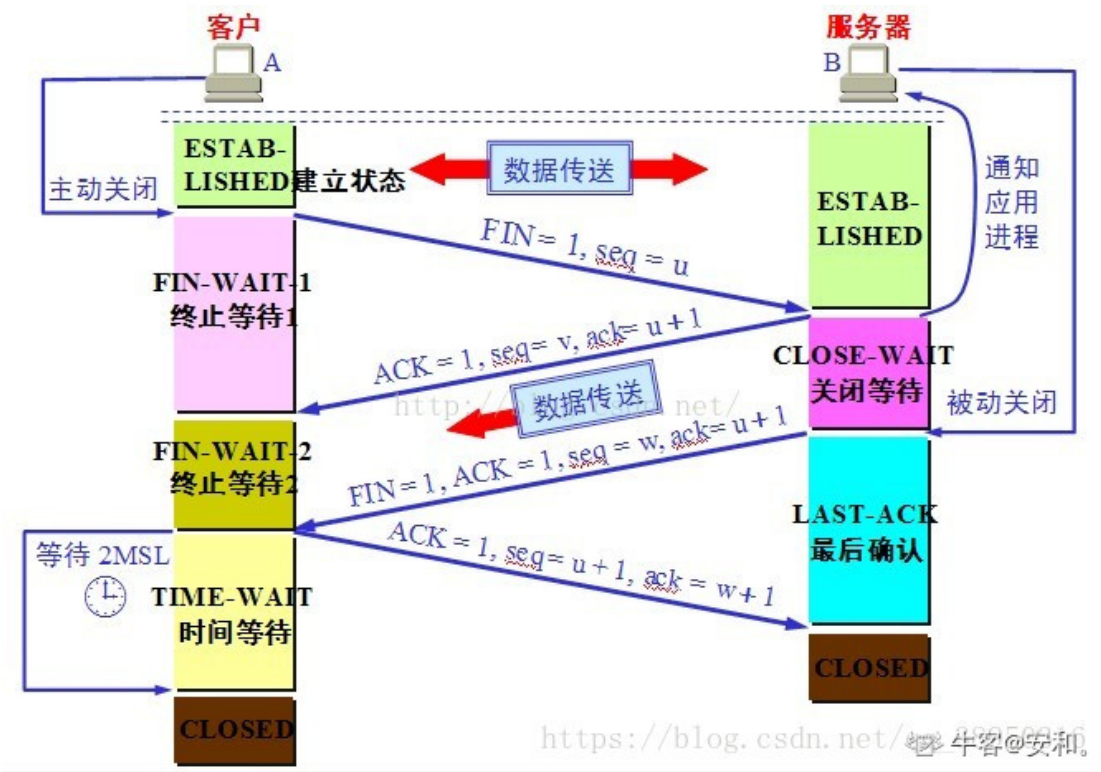
为丢包而源源不断地发送数据报，造成死锁。

2.16为什么服务器端易受SYN攻击？有什么防范措施吗？

(1) 因为服务器端的资源是在第二次握手时分配的，而客户端的资源是在第三次握手时才分配的，若服务器收到大量伪造的IP地址发来的SYN包，则需要不断回复并等待确认，由于等待不到确认这些连接就会占满未连接队列，导致后来正常的连接请求报文被丢弃，从而引起网络拥塞甚至系统瘫痪。

(2) 防范措施：降低主机的等待时间使主机尽快释放半连接的占用、短时间内收到某IP的重复连接请求报文则丢弃后续请求。

2.17简述四次挥手的过程。



(1) 第一次挥手：客户端发出连接释放报文（FIN=1，seq=u），进入FIN-WAIT-1状态。

(2) 第二次挥手：服务器收到连接释放报文后发出确认报文（ACK=1，ack=u+1，seq=v），进入CLOSE-WAIT状态。这时客户端向服务器方向的连接就释放了，这个连接处于半关闭状态，服务器还可继续发送数据。

(3) 中间状态：客户端收到服务器的确认报文后，进入FIN-WAIT-2状态，等待服务器发送连接释放报文，此时仍要接收数据。

(4) 第三次挥手：服务器最后的数据发送完，向客户端发送连接释放报文（FIN=1, ACK=1, ack=u+1, seq=w），进入LAST-ACK状态。

(5) 第四次挥手：客户端收到服务器的连接释放报文后，必须发出确认报文（ACK=1, ack=w+1, seq=u+1），进入TIME-WAIT状态。注意此时连接还未释放，必须进过2*MSL（最长报文寿命）的时间，客户端撤销相应的TCB后，才进入CLOSED状态。服务器一旦收到确认报文，立即进入CLOSED状态。

2.18为什么客户端最后还要等待2*MSL (Maximun Segment Lifetime) ？

(1) 第一，保证客户端发送的最后一个ACK报文能够到达服务器，因为这个ACK报文可能丢失，服务器等待2*MSL后，会重新发送连接释放报文。等待2*MSL保证如果确认报文丢失客户端可以收到重传的连接释放报文，然后再次发送确认报文。

(2) 第二，使本连接持续的时间内所产生的所有报文段都从网络中消失，不会出现类似“两次握手”的意外情况。

2.19为什么建立连接是三次握手，而关闭连接是四次挥手？

(1) 建立连接的时候，服务器在LISTEN状态下，不需要等待，可以立即建立连接，因此只需要三次握手。

(2) 而关闭连接时，服务器收到对方的FIN报文时，仅仅表示对方不再发送数据了但是还能接受数据，而自己未必将所有数据都发送完了，所以服务器可以立即关闭，也可以发送一些数据后再关闭，所以服务器的确认报文（ACK=1）和连接释放报文（FIN=1, ACK=1）一般分开发送，因此形成四次握手。

2.20 DNS服务器和客户机设置完毕后，有哪三个命令可以测试其设置是否正确？

(1) PING：是工作在TCP/IP网络体系结构中应用层的一个服务命令，主要是向特定的目的主机发送ICMP Echo请求报文，测试目的站是否可达及了解其有关状态。

(2) IPCONFIG：可用于显示当前TCP/IP配置的设置值。

(3) NSLOOKUP：用于查询DNS记录（nslookup domain [dns-server]）、查看域名解析是否正常、在网络故障的时候用来诊断网络问题。

2.21网络编程的作用？

网络编程的本质使用套接字来进行进程间通信，但这些进程是分布在不同主机上的进程，因此网络编程在物联网方面有比较大的作用。

2.22 socket网络编程中，哪五个元素可以明确标识一条连接？

五元组：客户端端口、客户端IP、服务端端口、服务端IP、传输协议。

2.23什么是TCP粘包？

TCP粘包就是指发送方发送的若干数据包到达接收方时粘成了一个包。从接收缓冲区来看，后一包数据的头紧接着前一包数据的尾。

2.24造成TCP粘包的原因？

(1) 发送方原因：TCP默认使用Nagle算法（主要作用：减少网络中报文段的数量），Nagle算法主要做两件事情：

①只有上一个分组得到确认，才会发送下一个分组。

②收集多个小分组，在一个确认到来时一起封装并发送。

(2) 接收方原因：TCP接收到数据包，应用层并不会立即处理。实际上，TCP将收到的数据包保存在接收缓存里，然后应用程序主动从缓存读取收到的分组。这样一来，如果TCP接收数据包到缓存的速度大于应用程序从缓存中读取数据包的速度，多个包就会被缓存，应用程序就有可能读取到多个首尾相接的包。

2.25什么时候需要处理粘包现象？

(1) 如果发送方发送的多组数据本来就是同一块数据的不同部分，比如说一个文件被分成多个部分发送，这时当然不需要处理粘包现象。

(2) 如果多个分组毫不相关，甚至是并列关系，那么就一定要处理粘包现象了。

2.26 如何解决粘包问题？

(1) 发送方：可以使用TCP_NODELAY选项来关闭Nagle算法。

(2) 接收方：接收方无法处理，只能将问题交给应用层来处理。

(3) 应用层：应用层的解决办法简单可行，不仅能解决接收方的粘包问题，还能解决发送方的粘包问题。

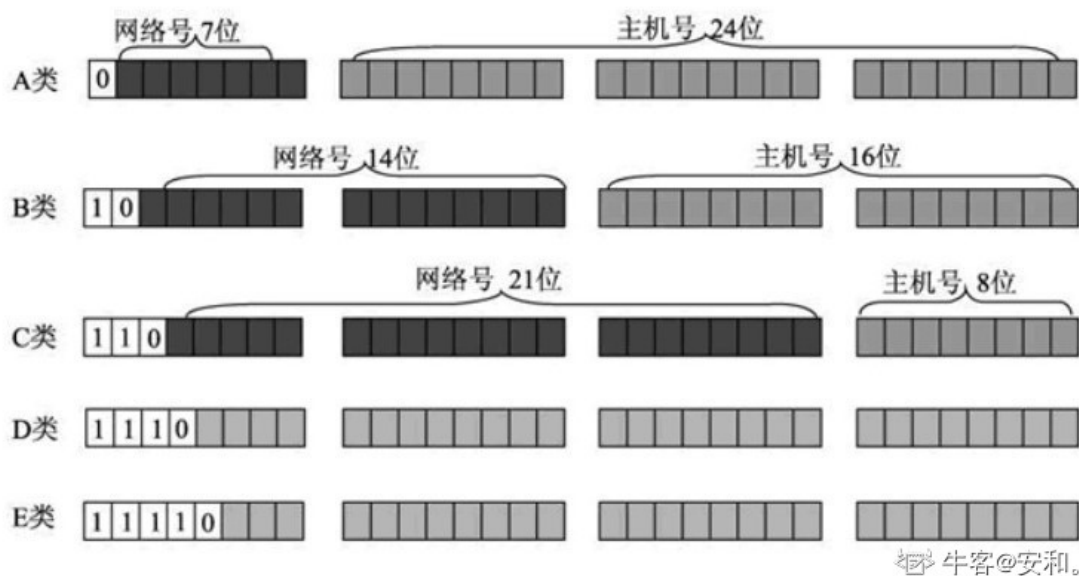
① 格式化数据：每条数据都有固定的格式（开始符、结束符），这种方法简单易行，但是选择开始符和结束符时一定要确保每条数据的内部不包含这两种符号。

② 发送数据包长度：发送每条数据时，将数据长度一并发送，例如规定数据的前四位是数据的长度，这样接收方应用层在处理时就可以根据长度来判断每个分组的开始和结束位置。

2.27 IP地址的分类？

答案：A类分配给政府机关单位使用，B类分配给中等规模企业使用，C类分配给任何需要的人使用，D类用于多播，E类用于实验。

解读：



(1) 由于每类地址的开头是固定的，因此每类地址都有自己的范围：

① A类：0.0.0.0 ~ 127.255.255.255。

②B类：128.0.0.0 ~ 191.255.255.255。

③C类：192.0.0.0 ~ 223.255.255.255。

④D类：224.0.0.0 ~ 239.255.255.255。

⑤E类：240.0.0.0 ~ 255.255.255.254。

(2) 在进行IP地址的分配时，有一些IP地址具有特殊含义，不会分配给互联网的主机。保留了一些IP地址用于私有网络，称为私有地址；保留一些IP地址用于测试，称为保留地址。

①A类私有地址范围：10.0.0.0 ~ 10.255.255.255；

A类保留地址范围：127.0.0.0 ~ 127.255.255.255。

②B类私有地址范围：172.16.0.0 ~ 172.31.255.255；

B类保留地址范围：169.254.0.0 ~ 169.254.255.255。

③C类地址私有范围：192.168.0.0 ~ 192.168.255.255。

2.28 IP地址与MAC地址的相同点和不同点？

(1) 相同点：都是唯一的。

(2) 不同点：

①分配依据不同：IP地址的分配是基于网络拓扑设计的，可以改动；MAC地址是基于制造商的，不能改动。

②长度不同：IP地址为32位，MAC地址为48位。

③寻址协议不同：IP地址应用于OSI第三层，即网络层；MAC地址应用于OSI第二层，即数据链路层。网络层协议可以使数据从一个网络传递到另一个网络（ARP根据目的IP地址，找到中间节点的MAC地址，通过中间节点传送，从而到达目的网络）；数据链路层协议可以使数据从一个节点传递到相同链路的另一个节点上（通过MAC地址）。

2.29 路由器、交换机、集线器的区别？

- (1) 路由器工作在网络层，根据网络层提供的信息（IP地址）来选择路由。
- (2) 交换机工作在数据链路层，通过数据链路层提供的信息（MAC地址）来选择端口。
- (3) 集线器工作在物理层，只对信号进行整形、放大后再重发（广播给其他所有端口）。

讨论

 评论



帖子还没人回复，快来抢沙发！

1.驱动开发 (13道)

1.1 Linux驱动程序的功能是什么？

- (1) 对设备初始化和释放。
- (2) 进行内核与硬件的数据交互。
- (3) 检测和处理设备出现的错误。

1.2内核程序中申请内存使用什么函数？

答案：kmalloc()、kzalloc()、vmalloc()。

解读：

(1) void *kmalloc(size_t size, gfp_t flags);

①申请连续的物理内存，这对于要进行DMA的设备十分重要，但大小不能超过128KB，其中有16B是被页描述符占用了。

②较常用的flag有GFP_ATOMIC（分配内存的过程是一个原子过程）、GFP_KERNEL（正常分配内存）、GFP_DMA（给DMA控制器分配内存）。

③对应的内存释放函数为void kfree(const void *objp)。

(2) void *kzalloc(size_t size, gfp_t flags);

①kzalloc()相对kmalloc()只是额外增加了__GFP_ZERO标志，除了申请内存外，还会对申请到的内存内容清零。

②对应的释放函数也是kfree()。

(3) void *vmalloc(unsigned long size);

①申请虚拟地址连续的内存空间，但其对应的物理内存不一定连续，因此对申请的内存大小没有限制。

②对应的内存释放函数为void free(const void *addr)。

③注意：vmalloc()和vfree()可以睡眠，因此不能在中断上下文调用。

1.3内核程序中申请内存和应用程序时申请内存有什么区别？

答案：内核中申请内存空间用的是函数kmalloc、kzalloc、vmalloc，应用程序申请内存用的函数是malloc。

解读：

- (1) kmalloc/kzalloc直接分配连续的物理地址（虚拟地址也是连续的）。
- (2) vmalloc分配连续的虚拟地址，但物理地址不一定连续。分配时实际分配了物理内存，不过这个物理内存页面是在公共的页表进行了映射，并没有在本进程的页表进行映射，当访问这段内存时，触发do_page_fault异常（缺页中断）才完成页表的同步工作。
- (4) malloc是用户空间申请内存的方法，分配连续的虚拟地址，物理地址一般不会连续。在分配时并没有做实际物理页的分配动作，实际分配物理页的动作是在do_page_fault异常（缺页中断）处理中完成的。

1.4自旋锁和信号量在互斥使用时需要注意什么？在中断服务程序里面的互斥是使用自旋锁还是信号量？

- (1) 使用自旋锁的进程不会睡眠，而使用信号量的进程会睡眠。
- (2) 中断服务程序使用的是自旋锁，原因是中断服务程序处于中断上下文，中断上下文是不参与调度的，也就没有保护现场与恢复现场，一旦睡眠就回不来了。

1.5驱动卸载异常可能是由什么原因引起的？

可能是因为有进程在使用该模块。

1.6 Linux中引入模块机制有什么好处？

- (1) 应用程序在退出时，可以不管资源的释放或者其他的清除工作，而把这些任务交给模块退出函数(exit)。
- (2) 模块机制有助于缩短模块的开发周期，因为模块的安装和卸载都很方便。

1.7 Linux设备驱动程序中，使用哪两个函数进行中断处理程序的注册和注销？

(1) 注册中断：

```
int request_irq(unsigned int irq, irqreturn_t (*handler)
(int, void *, struct pt_regs *), unsigned long flag, const char *dev_name, void *dev_id);
```

参数的意义依次是中断号，中断处理函数，中断管理有关掩码，中断请求设备名，中断信号线。

(2) 注销中断：

```
void free_irq(unsigned int irq, void *dev_id);
```

参数的意义分别是中断号和中断信号线。

1.8 写一个中断服务程序需要注意哪些地方？

(1) 中断服务程序应该尽可能短，把能放在底半部（tasklet、workqueue）的任务尽量放在底半部。

(2) 中断服务程序中不能有阻塞操作，因为中断期间是完全占用CPU的，不存在内核调度，中断被阻塞住，其他进程将无法推进。

(3) 中断服务程序的返回值要用操作系统定义的宏，而不能用自己定义的。

(4) 中断服务程序不能有不可重入的操作，如浮点数计算、printf()等。

1.9 Linux系统打开设备文件，进程可能处于三种基本状态，如果多次打开设备文件，驱动程序应该实现什么？

互斥。

1.10 简述static对于工程模块化的作用。

static可以让全局变量或函数的作用域限制在当前模块，不会与其他模块发生冲突。因为在嵌入式系统中，一个程序可能是很多程序员共同完成的，在定义变量及函数的过程中，可能会重命名，给系统集成带来麻烦。

1.11并发是什么？驱动里面为什么要有互斥控制？如何实现？

- (1) 并发是指多个执行单元同时、并行地被执行，而并发的执行单元对共享资源（硬件资源和软件上的全局变量、静态变量）的访问很容易导致竞态。
- (2) 解决竞态问题的途径是保证对共享资源的互斥访问。
- (3) 访问共享资源的代码区域被称为临界区，临界区需要用某种互斥机制加以保护，中断屏蔽、原子操作、信号量、自旋锁都是Linux设备驱动中可以采取的互斥机制。

1.12 Linux内核有哪些同步方式？

原子操作、信号量、自旋锁、读写锁、顺序锁等。

1.13在一个多任务嵌入式系统中，有一个CPU可直接寻址的32位寄存器REGn，地址为0x1F000010，编写一个安全的函数将寄存器REGn的指定位反转？

答案：

```
void bit_reverse(uint32_t nbit)
{
    *((volatile unsigned int *)0x1F000010) ^= (0x01 << nbit);
}
```

注意：

- (1) 指定位反转用异或^。
- (2) 由于是寄存器地址，因此强制类型转换的时候要加上volatile。

讨论

 评论



帖子还没人回复，快来抢沙发！

2.操作系统 (39道)

2.1 Linux系统的组成部分?

Linux内核、Linux文件系统、Linux shell、Linux应用程序。

2.2 Linux内核的组成部分?

(1) 第一种分类方式：内存管理子系统、进程管理子系统、文件管理子系统、I/O管理子系统。

(2) 第二种分类方式：进程调度 (SCHED)、进程间通信 (IPC)、内存管理 (MM)、虚拟文件系统 (VFS)、网络接口 (NET)。

2.3内存管理MMU的作用?

- (1) 地址映射。
- (2) 内存分配和回收。
- (3) 内存保护。
- (4) 内存扩充。

2.4常见的操作系统进程调度策略有哪些?

答案:

- (1) 先来先服务 (FCFS)。
- (2) 短进程优先 (SPF)。
- (3) 高响应比优先 (HRN)。
- (4) 时间片轮转 (RR)。
- (5) 多级反馈队列 (RRWMF)。
- (6) 最高优先权优先 (FPF)。

解读:

(1) 先来先服务 (FCFS)：每次调度是从就绪队列中选择一个最先进入该队列的进程，为之分配处理机，使之投入运行。该进程一直运行到完成或发生某事件而阻塞后才放弃处理机。

(2) 短进程优先 (SPF)：从就绪队列中选出一个估计运行时间最短的进程，将处理机分配给它，使它立即执行并一直执行到完成，或发生某事件而被阻塞放弃处理机时再重新调度。

(3) 高响应比优先 (HRN)：在短作业优先的基础上，使作业的优先级随着等待时间的增加而以速率 a 提高，则长作业在等待一定的时间后，必然有机会分配到处理机。

(4) 时间片轮转 (RR)：将所有的就绪进程按先来先服务的原则排成一个队列，每次调度时，把CPU分配给队首进程，并令其执行一个时间片，时间到了之后将它送往

就绪队列的末尾。接着调度下一个队首进程.....如此循环。

(5) 多级反馈队列 (RRWMF) :

①设置多个就绪队列，并为各个队列赋予不同的优先级。第一个队列的优先级最高，第二个队列次之，其余各队列的优先权逐个降低。该算法赋予各个队列中进程执行时间片的大小也各不相同，在优先权愈高的队列中，为每个进程所规定的执行时间片就愈小。例如，第二个队列的时间片要比第一个队列的时间片长一倍，第 $i + 1$ 个队列的时间片要比第 i 个队列的时间片长一倍。

②当一个新进程进入内存后，首先将它放入第一队列的末尾，按FCFS原则排队等待调度。当轮到该进程执行时，如它能在该时间片内完成，便可准备撤离系统；如果它在一个时间片结束时尚未完成，调度程序便将该进程转入第二队列的末尾，再同样地按FCFS原则等待调度执行；如果它在第二队列中运行一个时间片后仍未完成，再依次将它放入第三队列，如此下去，当一个长进程从第一队列依次降到第 n 队列后，在第 n 队列便采取按时间片轮转的方式运行。

③仅当第一队列空闲时，调度程序才调度第二队列中的进程运行；仅当第 $1 \sim (i-1)$ 队列均空时，才会调度第 i 队列中的进程运行。如果处理机正在第 i 队列中为某进程服务时，又有新进程进入优先权较高的队列(第 $1 \sim (i-1)$ 中的任何一个队列)，则此时新进程将抢占正在运行进程的处理机，即由调度程序把正在运行的进程放回到第 i 队列的末尾，把处理机分配给新到的高优先权进程。

(6) 最高优先权优先 (FPF) : 把处理机分配给就绪队列中优先权最高的进程，这时，又可进一步把该算法分成如下两种：

①非抢占式优先权算法：系统一旦把处理机分配给就绪队列中优先权最高的进程后，该进程便一直执行下去，直至完成；或因发生某事件使该进程放弃处理机时，系统方可再将处理机重新分配给另一优先权最高的进程。这种调度算法主要用于批处理系统中，也可用于某些对实时性要求不严的实时系统中。

②抢占式优先权调度算法：系统同样是把处理机分配给优先权最高的进程，使之执行。但在其执行期间，只要又出现了另一个其优先权更高的进程，进程调度程序就立即停止当前进程（原优先权最高的进程）的执行，重新将处理机分配给新到的优先权最高的进程。

2.5满足短作业优先又不会发生饥饿现象的调度算法是？

高响应比优先。

2.6 I/O子系统层次结构

从上到下依次是用户层I/O软件，设备独立性软件，设备驱动程序，中断处理程序。

2.7逻辑地址、线性地址、物理地址、总线地址、虚拟地址的区别？

- (1) 逻辑地址：与内存段相关的偏移地址部分。例如，在C语言获取指针变量的值，就是逻辑地址，不和绝对的物理地址相关。
- (2) 线性地址：线性地址 = 逻辑地址 + 基地址。
- (3) 物理地址：如果启用了分页机制，那么线性地址使用页表项变换后就是物理地址。如果没有启用分页机制，那么线性地址就是物理地址。
- (4) 总线地址：x86下的I/O地址，ARM下的物理地址。（在x86下，外设的I/O地址是独立的，即有专门的指令访问外设I/O，I/O地址就是“总线地址”，而RAM地址就是“物理地址”。在ARM下，I/O和RAM统一编址，但linux为了统一各个平台，仍然保留这个概念，总线地址其实就是物理地址。）
- (5) 虚拟地址：MMU虚拟出来的地址。

2.8操作系统的内存分配一般有哪几种方式，各有什么优缺点？

- (1) 分页存储管理：优点是不需要连续的内存空间，且内存利用率高（只有很小的页内碎片）；缺点是不易于实现内存共享与保护。
- (2) 分段存储管理：优点是易于实现段内存共享和保护；缺点是每段都需要连续的内存空间，且内存利用率较低（会产生外部碎片）。
- (3) 段页式存储管理：优点是不需要连续的内存空间，内存利用率高（只有很小的页内碎片），且易于实现段内存共享和保护；缺点是管理软件复杂性较高，需要的硬件以及占用的内存也有所增加，使得执行速度下降。

2.9用户空间与内核通信的方式有哪些？

系统调用接口 SCI (API)	用户空间进程通过系统调用进入内核空间，访问指定的内核空间数据。
驱动程序	用户空间进程可以使用封装后的系统调用接口访问驱动设备节点，和运行在内核空间的驱动程序通信。
copy_to_user()、 copy_from_user()	在驱动程序中调用的接口，实现用户空间与内核空间的数据拷贝操作，应用于实时性要求不太高的项目中。

procfs	是一种特殊的伪文件系统，是Linux内核信息的抽象文件接口，大量内核中的信息以及可调参数都被作为常规文件映射到这个目录树中，用户可通过echo或cat这样的命令对系统信息进行修改和查询。
共享内存mmap	在代码中调用接口，实现内核空间与用户空间的地址映射，在实时性要求很高的项目中为首选。
netlink	用户进程使用标准的socketAPI就可以使用netlink提供的强大功能。

2.10调用API read()/write()时，内核具体做了哪些事情？

用户进程调用read()/write()后进入内核空间 -> 调用内核的sys_read()/sys_write()函数
-> 内核函数进一步调用fop.read()/fop.write()指向的函数，也就是驱动程序里具体的读写函数

2.11系统调用的作用？

- (1) 为应用程序提供访问硬件资源的统一接口，以至于应用程序不必关心具体的硬件操作细节。
- (2) 对系统内核进行保护，保证系统的稳定和安全，因为系统调用规定了用户进程进入内核的具体方式以及所能访问的数据范围。

2.12 BootLoader、Linux内核、根文件系统的关系？

- (1) 系统一上电，就执行BootLoader来初始化处理器及外设，将系统的软硬件环境带到一个合适的状态。
- (2) 然后将Linux内核、根文件系统从非易失性存储器读取到内存中并执行Linux内核。
- (3) Linux内核完成初始化之后就挂载某个文件系统作为根文件系统。

2.13 BootLoader的启动过程大多可以分为哪两个阶段？

(1) 第一阶段是汇编的，完成一些依赖于CPU体系结构的初始化，并调用第二阶段的代码。

(2) 第二阶段是C语言的，进一步完成硬件初始化，建立内存映射，并将内核映像和根文件系统读取到内存中，然后调用内核。

2.14 Linux下检查内存状态的命令？

ps -aux命令、free命令、vmstat命令、top命令、cat/proc/meminfo命令。

2.15在Linux里面用gcc编译程序时，分为哪四个阶段？

(1) 预处理：引入头文件、进行宏替换、处理条件编译指令、去除注释、添加行号。

(2) 编译：进行语法分析等，并生成汇编代码.s。

(3) 汇编：将汇编代码转成二进制代码.o（目标文件）。

(4) 链接：将有关的目标文件彼此连接为可执行代码。分为静态链接（将库文件代码搬迁到可执行文件中，后缀是.a）和动态链接（在执行的时候转到库文件代码执行，后缀是.so）。

2.16什么是死锁？产生死锁的原因是什么？

(1) 死锁是指多个进程因竞争资源而造成的一种僵局（互相等待），若无外力作用，这些进程将无法向前推进。

(2) 原因：①系统资源不足。②资源分配不当。③进程推进的顺序不合适。

2.17死锁的四个必要条件是什么？

(1) 互斥条件：一个资源每次只能被一个进程使用，其他进程只能等待。

(2) 请求与保持条件：进程已经获得至少一个资源，但又提出了新的资源请求，而该资源已被其他进程占有，此时该进程被阻塞，但对已获得资源依然保持不放。

(3) 不可剥夺条件：进程所获得的资源不能被其他进程剥夺，只能被自己释放。

(4) 循环等待条件：若干进程形成首尾相接循环等待资源的关系。

注意：以上四个条件缺一不可。

2.18死锁的处理方法？

(1) 预防死锁：通过设置一些限制条件，去破坏产生死锁的必要条件。

(2) 避免死锁：在资源分配过程中，使用某种方法避免系统进入不安全的状态，从而避免发生死锁。

(3) 检测和解除死锁：允许死锁的发生，但是通过系统的检测之后，采取一些措施，将死锁清除掉。

2.19如何预防死锁？

(1) 破坏“请求与保持条件”：

①静态分配，即每个进程在开始执行时就申请它所需要的全部资源：

②动态分配，即每个进程在申请所需要的资源时它本身不占用系统资源。

(2) 破坏“不可剥夺条件”：一个进程在阻塞等待期间，其占有的资源被隐式释放后被其他进程使用，而阻塞等待的资源只有获得所有需要的资源才能重新启动。

(3) 破坏“循环等待条件”：采用资源的有序分配，将所有资源进行编号，紧缺的资源采用比较大的编号，一个进程只有获得较小编号的资源才可以申请较大编号的资源。

2.20硬链接与软链接的区别？

(1) 链接：是给系统中已有的某个文件指定另外一个可用于访问它的名称，链接也可以指向目录。即使我们删除这个链接，也不会破坏原来的文件或目录。

(2) 硬链接：引用的是文件在文件系统中的物理索引（inode），当你移动或删除原文件时，硬链接不会被破坏。硬链接只能引用同一文件系统中的文件。

(3) 软链接：就是新建一个文件（inode），这个文件专门用来指向别的文件（类似Windows中的快捷方式），若移动或删除原文件，则相应的软链接不可用。软链接可以跨文件系统，也可以对一个不存在的文件名或目录名进行链接。

2.21 虚拟内存是什么？

虚拟内存是内存管理的一种技术，使得应用程序认为自己拥有连续的内存空间，而实际上它对应的物理内存是不连续的，甚至还有一部分对应着外部磁盘存储器，在需要时再进行数据交换。

2.22 内存零头、外零头、内零头？

- (1) 内存零头：操作系统在内存分配时产生的一些无法利用的内存碎片。
- (2) 外零头：内存中存在着一些内存碎片，但这些内存碎片不归任何进程所有。段式存储管理会产生外零头，其中段的大小是程序逻辑确定的，大小不定。
- (3) 内零头：内存中存在着一些内存碎片，而这些内存碎片是归属于某个进程的。页式存储管理会产生内零头，其中页的大小是由系统确定的，大小固定。

2.23 64位的计算机有哪些优点？

- (1) 可以进行更大范围的整数计算；
- (2) 可以支持更大的内存，虚拟内存空间大小一般为 2^{48} （256TB）。64位的Linux一般使用48位表示虚拟内存空间地址，40位表示物理内存地址。

2.24 中断分为哪两种？

- (1) 异步中断：也叫外部中断，由CPU外设产生的电信号，其发生时间不可预估。其引入是为了支持CPU和设备之间的并行操作。
- (2) 同步中断：也叫内部中断、异常，由CPU内部产生的电信号，其特点是当前指令执行完才产生中断。其引入是为了表示CPU执行时本身出现的问题。

2.25 Linux系统的挂起、待机、休眠？

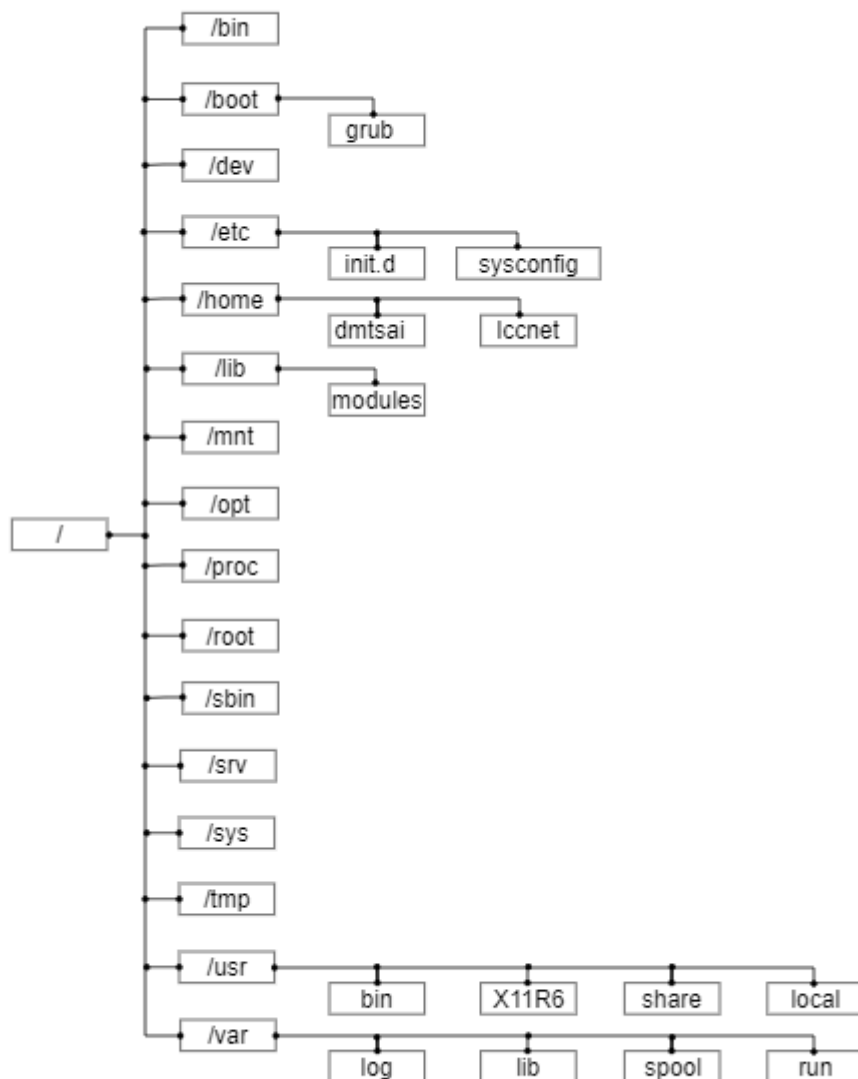
- (1) 挂起：CPU、内存工作，而硬盘、显示器等外部设备停止工作。
- (2) 待机：只对内存供电，CPU、外设停止工作。
- (3) 休眠：CPU、内存停止工作，内存中的数据保存在硬盘中，外部设备也停止工作。

2.26关机、重启命令？

- (1) 关机命令：halt、shutdown -h 时间、init0、poweroff。
 - (2) 重启命令：reboot、shutdown -r 时间、init6。
- 注意：shutdown可以安全地关闭或重启Linux系统，它在系统关闭之前给系统上所有登录用户提示一条警告信息，该命令还允许用户指定一个时间参数。

2.27 Linux目录结构，/usr、/tmp、/etc目录的作用？

- (1) Linux目录结构：



牛客@安和。

(2) /usr: 是Unix Software Resource的缩写，也就是Unix操作系统软件资源所放置的目录，而不是用户数据。

(3) /tmp: 让一般使用者或者是正在执行的程序暂时放置档案的地方。

(4) /etc: 系统主要的设定档案几乎都放置于这个目录，例如人员的账号密码档案、各种服务器的起始档案。一般是供给使用者查阅的，只有root用户有权限修改。FHS建议不要放置可执行档案在这个目录中。

2.28简述实时操作系统和非实时操作系统的区别？

(1) 实时操作系统：是可抢占式内核，响应外部事件尽可能快且响应时间具有确定性。有硬实时操作系统和软实时操作系统，硬实时操作系统的抖动（时间偏差）比软实时操作系统小。

(2) 非实时操作系统：非可抢占式内核，响应外部事的时间没有保证。

2.29常用的实时操作系统有哪些？非实时呢？

(1) 实时：Vxworks、 μ COS、eCOS、QNX、MQX、Nucleus、FREERTOS、RTLinux、RTEMS、RT-Thread。

(2) 软实时或非实时：WinCE、Linux。

2.30 Linux内核的调度策略有哪些？

(1) SCHED_OTHER分时调度策略；

(2) SCHED_FIFO实时调度策略，先到先服务。

(3) SCHED_RR实时调度策略，时间片轮转。

2.31扇区、簇、块、页的概念？

(1) 扇区：顾名思义，每个磁盘有很多条类似同心圆的磁道，磁道被分割成多个部分，每部分的弧长加上到圆心的两条半径，就形成一个扇区。是磁盘读写的基本单位。

(2) 簇：每个簇可以包括 2^n 个扇区。

(3) 块：操作系统中最小的逻辑存储单位，即与磁盘打交道的最小单位。

(4) 页：内存的最小存储单位，大小为 2^n 个块的大小，即操作系统与内存打交道的最小单位。

2.32简述处理器在读内存过程中，CPU、MMU、cache、内存如何协同工作？

(1) CPU内核发出VA向TLB请求页表项，它缓存最近查找过的VA对应的页表项，如果TLB中刚好缓存了该VA对应的页表项则直接返回给CPU；否则就由MMU去物理内存中读出页表项保存在TLB中，同时返回给CPU内核。（TLB可以减少访问物理内存的次数）

(2) MMU首先检查页表项中的权限位，如果没有访问权限，就引发一个异常给CPU内核；有权限则检查是否允许缓存（页表项中不仅保存着物理页面的基地址，还保存着权限和是否允许cache的标志）。

(3) 如果不允许cache则MMU直接发出PA从物理内存中读取数据到CPU内核。

(4) 如果允许cache，则MMU以VA为索引到cache中查找是否缓存了要读取的数据，如果是则直接返回给CPU内核；如果不是则MMU发出PA从物理内存中读取数据并缓存到cache中，同时返回给CPU内核。

2.33 ARM CPU上的地址转换涉及哪三个概念？

(1) 虚拟地址 (VA)：CPU内核对外发出VA。

(2) 变换后的虚拟地址 (MVA, Modified Virtual Address)：VA被变换为MVA供cache和MMU使用，由硬件自动完成。如果 $VA < 32M$ ，则需要使用PID来转换为 $MVA(VA | (PID \ll 25))$ ，目的是当两个进程的虚拟地址空间有重叠时，把重叠的VA映射到不同的PA上去，减少切换进程的代价，

(3) 物理地址 (PA)：最后使用PA读写实际设备。

2.34 MMU的和不用MMU的操作系统有哪些？带MMU的和不带MMU的CPU有哪些？

(1) 用MMU的操作系统有:Windows、MacOS、Linux、Android；不用MMU的操作系统有：FreeRTOS、VxWorks、UCOS。

(2) 带MMU的CPU有：Cortex-A系列、ARM9、ARM11系列；不带MMU的CPU有：Cortex-M系列（STM32就是使用M系列内核）。

2.35为什么uboot要关掉cache？

cache主要用来加快CPU读取内存中指令的速度，但在CPU初始化时，内存并未准备就绪，直接进行数据读取，势必导致指令取址异常。

2.36内核函数mmap的实现原理？

答案：mmap函数能够实现把一个文件映射到一个内存区域，使得我们可以像读写内存一样读写文件，比单纯地调用read/write函数要快上许多。我们可以把内存中的内容拷贝到一个文件中实现内存备份，也可以把文件映射到内存来恢复某些服务。

解读：实现共享内存是mmap函数的主要应用之一，mmap系统调用使得进程之间通过映射同一个普通文件实现共享内存，是一种进程间通信方式。

2.37缺省的Linux系统中，从后台启动进程，应在命令的结尾加上&。

2.38 ps跟top有什么区别？

- (1) ps看到的是命令执行瞬间的进程信息；而top可以实时地持续监视，不断更新进程状态。
- (2) ps只是查看进程信息，而top还可以监视系统性能，如CPU负载、内存消耗。
- (3) 另外top还可以操作进程，如改变优先级（命令r）和关闭进程（命令k）。

2.39请写出下列常用命令？

Linux查看线程ID	①ps -T -p 进程号 ②top -H -p 进程号 ③pstree -Aup -h 进程号
Linux设置文件目录权限	chmod
uboot查看环境变量	printenv

讨论

 评论



帖子还没人回复，快来抢沙发！